

Manuale tecnico

Riproduzione di *Audio Sparse-Transformer for Speech Classification*

Kavaki & Mandel, ICASSP 2025

Progetto d'esame — Bo31278 Deep Learning, Fall 2025

MSc in Artificial Intelligence, Università di Firenze

Versione 1.6 — 20 agosto 2026 · documento vivo, aggiornato a ogni sessione

v1.1: consultato il codice di SparseFormer e di EAT. Corrette le formule di `mixing` e `phasemix`, la sequenza dello stem e il default della normalizzazione; risolte la formula degli offset e l'inizializzazione a griglia.

v1.2: letti `trainer.py` e `helper_funcs.py` di EAT. Chiusi i parametri di `one-cycle`, la programmazione delle augmentation e il criterio del weight decay; corretto il target mixato in multi-hot con soglia; chiarita la giustificazione del mel.

v1.3: nuovo capitolo 12 con il primo run completo (94,67%), la saturazione della curva ben prima delle 100 epoche, il comportamento dell'EMA lungo lo schedule e il profilo fonologico degli errori; corretta l'osservazione sulla norma del gradiente.

v1.4: implementato l'estrattore sparso; i suoi FLOPs sciolgono la contraddizione «96 vs 196 kernels» a favore di `c96` (§12.7-12.8).

v1.5: il metodo sparso non è deterministico su GPU (§12.9); diagnostica sul movimento delle regioni (§12.10); il guadagno in FLOPs non si traduce in velocità (§12.11).

v1.6: EAT documentato per intero (§1.4) e panorama della letteratura su Speech Commands (§1.5); §10 completato con tutti i moduli e script; rinumerate le sezioni del cap. 11.

COME LEGGERE QUESTO MANUALE

Documenta **tutto**: la matematica sottostante, l'architettura, ogni scelta implementativa e la sua motivazione. È pensato per due usi: ripassare in vista dell'orale, e poter rispondere a «perché hai fatto così» su qualunque riga di codice.

Tre marcatori ricorrono nel testo:

DAL PAPER dichiarato esplicitamente da Kavaki & Mandel · **ASSUNZIONE** scelta nostra su un punto in cui il paper tace · **EVIDENZA** supportato da una misura che abbiamo eseguito.

Documenti fratelli: `Pipeline_progetto_DL.pdf` (il piano) e `Diario_di_bordo.pdf` (la cronaca, con gli errori).

INDICE

1. Il problema e l'inquadramento — keyword spotting, i due paper assegnati, EAT in dettaglio, il panorama su Speech Commands, la catena di ancestors
2. Dalla forma d'onda allo spettrogramma — STFT, scala mel, logaritmo, normalizzazione
3. Il transformer — attenzione, scaling, complessità, pre-norm, codifica posizionale
4. Convoluzione e early convolution — aritmetica, campo recettivo, perché serve
5. Ottimizzazione — da SGD ad AdamW, one-cycle, EMA
6. Loss e augmentation — softmax, BCE, mixup, phasemix
7. Il metodo sparso, per intero — regioni, campionamento, decoding adattivo
8. La ricostruzione del modello denso — vincoli numerici, ricerca, risultati
9. Catalogo delle assunzioni — ognuna con la sua evidenza
10. Il codice, file per file — tutti i 19 moduli e script
11. Protocollo sperimentale e misura
12. **Risultati** — primo run completo, saturazione della curva, profilo degli errori
13. Riferimenti

1 · Il problema e l'inquadramento

1.1 Che cosa stiamo classificando

Il compito è **keyword spotting**: data una registrazione di un secondo contenente una singola parola pronunciata, dire quale delle 35 parole del vocabolario è. Il dataset è *Google Speech Commands V2* (Warden 2018): 105 829 clip a 16 kHz.

È un problema di classificazione a 35 classi, con due caratteristiche che ne determinano il trattamento. Primo: gli esempi sono **corti e di lunghezza fissa**, quindi non serve gestire sequenze di lunghezza variabile né allineamento. Secondo: il compito è pensato per **dispositivi a risorse limitate** — un assistente vocale che ascolta in continuo — e per questo il costo computazionale è una metrica di prima classe, non un dettaglio.

1.2 I due paper e il loro rapporto

Gazneli et al. 2022 — EAT. Rete end-to-end che opera sulla *forma d'onda grezza*, senza spettrogramma: stack di convoluzioni 1D seguito da un encoder transformer. Il contributo dichiarato non è l'architettura ma un insieme di *augmentation audio*. EAT-S ha 5,3 M parametri e riporta 98,15 % su Speech Commands.

Kavaki & Mandel 2025 — Audio Sparse-Transformer. Osserva che la complessità della self-attention cresce quadraticamente con la lunghezza della sequenza, e che nell'audio le dipendenze a lungo raggio contano poco. Sostituisce quindi la lunga sequenza di frame temporali con una **sequenza cortissima di token latenti** — quattro — ottenuti campionando poche regioni informative dello spettrogramma. Riporta 96,88 % con 2,87 M parametri e 0,055 G FLOPs.

IL PUNTO CHE HA RISTRUTTURATO IL NOSTRO PIANO

Il confronto *controllato* del paper non è contro EAT-S, ma contro il proprio **dense-transformer**: la stessa identica architettura che processa la sequenza dei frame temporali invece dei token latenti (96,67 %, 4,80 M, 0,645 G). Il numero di EAT-S è **citato dalla letteratura**, non rimisurato dagli autori.

Conseguenza: il baseline da riprodurre per primo è il denso, non EAT-S. Condivide con lo sparso tutto il codice a valle e isola i bug del training loop da quelli del tokenizer.

1.3 La catena di ancestors

Il metodo non nasce nell'audio. È la trasposizione di **SparseFormer** (Gao et al. 2023), un modello per ImageNet che riconosce immagini campionando poche regioni invece di processare tutti i patch. Da SparseFormer arrivano tre componenti, ciascuna con la propria origine:

Componente	Origine
Parametrizzazione delle regioni	Faster R-CNN (Ren et al. 2015) — la codifica (Δx , Δy , $\log \Delta w$, $\log \Delta h$) delle bounding box
Decoding adattivo	AdaMixer (Gao et al. 2022) — pesi di mixing generati dalla query invece che fissi
Front-end convolutivo	Xiao et al. 2021 — <i>Early Convolutions Help Transformers See Better</i>

Leggere SparseFormer non è opzionale: le cinque pagine ICASSP comprimono in tre formule ciò che lì è esposto per esteso.

1.4 EAT in dettaglio — l'altro paper assegnato

EAT è assegnato allo stesso titolo di Kavaki & Mandel, e all'esame va saputo per intero, non solo come fornitore di augmentation. La sua tesi è **opposta** a quella del filone dominante, ed è questo che lo rende interessante da mettere accanto all'altro.

LA TESI: END-TO-END CONTRO RAPPRESENTAZIONI TEMPO-FREQUENZA

Il riconoscimento audio si basa quasi ovunque sul trasformare il segnale in uno spettrogramma mel e poi trattarlo come un'immagine. EAT rifiuta questo passaggio e opera sulla **forma d'onda grezza**, con due argomenti:

- il mel-spettrogramma «comes at the cost of having to carefully adjust the parameters for time-frequency resolution and compression» — cioè introduce iperparametri che vanno tarati per ogni compito, ed è esattamente il problema in cui ci siamo imbattuti noi (§2.5, dove il paper sparso non li dichiara affatto);
- l'assenza di pre-processing «streamlines the system, by requiring fewer parameters to tune, and facilitates shifting between tasks with distinct signal content».

EAT riconosce però che i sistemi basati su mel *vincono empiricamente*, e cita una comparazione sistematica fra rappresentazioni che ne deduce la superiorità. La sua scommessa è che il divario si possa colmare con architettura e augmentation invece che con la rappresentazione.

L'ARCHITETTURA

Stack di convoluzioni 1D sulla forma d'onda — depth-wise con kernel lunghi sull'asse temporale, poi 1×1 channel-wise, filtro passa-basso fisso, downsampling di un fattore ≈ 256 — seguito da un encoder transformer e da una testa lineare. Due varianti:

Variante	Canali	Layer / teste	Embedding	Parametri
EAT-S	16	4 / 8	128	5,3 M
EAT-M	32	6 / 16	256	25,5 M

La struttura è concettualmente parallela a quella di Kavaki & Mandel — front-end convolutivo più transformer — ma il front-end lavora in una dimensione invece che in due, e sostituisce lo spettrogramma invece di prepararlo.

IL CONTRIBUTO VERO: LE AUGMENTATION

Il paper è esplicito nel collocare qui il proprio contributo, non nell'architettura. Due famiglie:

- **Label-mixing** — *mixup* (che è in realtà il between-class learning di Tokozume et al., §6.3.1), *FreqMix* (scambio di bin FFT fra due campioni a un cutoff), *PhaseMix* (§6.3.2), *TimeMix/CutMix* in versione 1D;
- **Label-preserving** — gain, rumore bianco e colorato, time shift, filtraggio, inversione di polarità, time masking, quantizzazione, resampling.

Kavaki & Mandel ne adottano **due sole** — *mixing* e *phasemix* — insieme all'intera ricetta di ottimizzazione: stesso AdamW con $\beta=(0,9 \cdot 0,99)$, stesso lr $3 \cdot 10^{-4}$, stesso one-cycle, stesso weight decay 10^{-5} , stesso EMA 0,995. È questa aderenza che ci ha autorizzato a ricavare da EAT i dettagli che il paper sparso tace (§5.2, §5.3, §6.3).

RISULTATI RIPORTATI

Dataset	Modello	Risultato
ESC-50 (from scratch)	EAT-S	92,15 %
ESC-50 (pretrain AudioSet)	EAT-M	96,3 %

Dataset	Modello	Risultato
UrbanSound8K (from scratch)	EAT-S	85,5 %
Speech Commands	EAT-S	98,15 %
AudioSet	EAT-M	42,6 mAP

IL RUOLO DI EAT NEL NOSTRO PROGETTO

Kavaki & Mandel lo usano come **termine di paragone citato**, non rimisurato: il 98,15 % e gli 0,373 G di EAT-S nella loro Tabella 2 vengono dal paper di EAT, non da un loro esperimento. Per noi EAT è quindi **due cose**: la fonte della ricetta di training e delle augmentation, e il riferimento di alto livello contro cui il metodo sparso si posiziona.

Riprodurre EAT non è richiesto ed è classificato come opzionale nel piano. L'unica ragione per farlo sarebbe rimisurare i suoi 0,373 G col *nostro* contatore, per confrontarsi con EAT-S senza mescolare convenzioni — ma quella calibrazione l'abbiamo già ottenuta per altra via (§8.4).

1.5 Il panorama su Speech Commands

La presentazione richiede di «introduce and motivate the problem being addressed in the context of the relevant literature». Questi sono i modelli che il paper cita, e cosa fa ciascuno:

Modello	Come tratta l'audio	Acc.	Params	Nota
KWT Berg 2021	Patch di mel-spettrogramma <i>non sovrapposti</i> , alla ViT, con class embedding	97,7 %	—	Il più vicino al nostro compito: keyword spotting con transformer
AST Gong 2021	Patch 16×16 <i>sovrapposti</i> di spettrogramma 2D	98,1 %	—	Il migliore, ma con pretraining ImageNet e augmentation pesanti
HTS-AT Chen 2022	Patch con struttura <i>gerarchica</i> : i token si riducono di 2^{i-1} per encoder	98,0 %	28,8 M	Riduce token progressivamente, ma resta grande e costoso
EAT-S Gazneli 2022	Forma d'onda grezza, conv 1D	98,15 %	5,3 M	Nessuno spettrogramma
SimPF Liu 2023	Front-end di pooling semplificato	95,6 %	3,4 M	Efficienza, a prezzo di accuratezza

Il posizionamento di Kavaki & Mandel. Tutti i modelli sopra processano l'audio in modo *denso e uniforme*: ogni patch, ogni frame, ogni campione. HTS-AT è l'unico che riduce i token, ma lo fa gerarchicamente e a posteriori. La proposta è di rompere questo schema alla radice — **non processare tutto e poi ridurre, ma decidere fin dall'inizio dove guardare**, con un numero di token fissato a priori e indipendente dalla lunghezza dell'ingresso.

Il compromesso dichiarato è chiaro: 96,88 % contro il 98,1 % dei migliori, cioè **circa un punto e mezzo in meno**, a fronte di un costo che è un settantacinquesimo di HTS-AT e un settimo di EAT-S. La tesi non è «siamo i più bravi» ma «si può buttare via quasi tutto il calcolo pagando poco».

1.6 Il codice di riferimento: cosa abbiamo consultato, e per cosa

La consegna chiede di reimplementare il metodo *in modo autonomo*. La pagina del corso prevede però esplicitamente che «reading other ancestors in the citation graph may be necessary», specialmente «when dealing with the details of the experimental procedures». Questa sezione dichiara esattamente cosa è stato consultato e a quale scopo. **Nessuna riga è stata copiata.**

Paper	Repository	Uso
Kavaki & Mandel 2025	nessuna	Verificato leggendo tutte e cinque le pagine: nessun link, nessuna menzione di codice pubblico. Il metodo è reimplementato dalle sole formule del paper.
SparseFormer Gao et al. 2023	github.com/showlab/sparseformer file: imagenet/models/sparseformer.py	Sciolte tre ambiguità che il paper ICASSP lascia aperte: la formula della normalizzazione «a tre deviazioni standard» (§7.3), l'inizializzazione di token e regioni (§7.1), e la sequenza esatta dello stem convolutivo (§4.3).
EAT Gazneli et al. 2022	github.com/Alibaba-MIIL/ AudioClassification file: datasets/batch_augs.py , trainer.py , utils/helper_funcs.py	Ricavate le formule di <code>mixing</code> e <code>phasemix</code> (§6.3), la loro programmazione e la costruzione del target mixato (§6.3.4–5), i parametri di one-cycle (§5.3) e il criterio del weight decay (§5.2).

PERCHÉ QUESTA CONSULTAZIONE ERA NECESSARIA

Una prima versione delle augmentation, scritta dalle sole descrizioni testuali, è risultata **sbagliata su quasi tutti i dettagli**: trasformata sbagliata, ampiezza mescolata invece che preservata, distribuzione di λ sbagliata, peso dell'etichetta sbagliato. Nessuno di questi errori era rilevabile dal testo del paper.

Attenzione alla versione: il repository di SparseFormer contiene anche una libreria `sparseformer/` corrispondente alle versioni ICLR 2024 e CVPR 2024, *successive e diverse*. Il file rilevante è quello sotto `imagenet/` , che implementa la v1 (arXiv:2304.03768), cioè la versione che Kavaki & Mandel citano.

2 · Dalla forma d'onda allo spettrogramma

2.1 Perché non lavorare sul segnale grezzo

Una clip di un secondo a 16 kHz è un vettore di 16 000 campioni. Un transformer che trattasse ogni campione come token affronterebbe una sequenza di lunghezza 16 000, con una matrice di attenzione da $2,56 \cdot 10^8$ elementi per testa: impraticabile. Serve una rappresentazione che comprima il tempo preservando il contenuto fonetico. EAT risolve il problema con convoluzioni 1D a stride elevato; Kavaki & Mandel usano invece la rappresentazione tempo-frequenza classica.

2.2 Short-Time Fourier Transform

Si divide il segnale in finestre brevi e sovrapposte, su ciascuna si calcola la trasformata di Fourier discreta. Detto $x[n]$ il segnale e $w[n]$ una finestra di lunghezza L :

$$X[m, k] = \sum_{n=0}^{L-1} x[n + m \cdot H] \cdot w[n] \cdot e^{-j2\pi kn/L} \quad (2.1)$$

dove m indicizza il frame temporale, k il bin di frequenza e H è l'*hop*, cioè l'avanzamento fra finestre consecutive.

Il compromesso fondamentale è quello di Heisenberg per l'analisi tempo-frequenza: una finestra lunga dà buona risoluzione in frequenza ($\Delta f \approx f_s/L$) ma cattiva nel tempo; una corta il contrario. Per il parlato la convenzione è una finestra di **20–30 ms**, che è la scala su cui l'apparato fonatorio si può considerare stazionario, con hop di **10 ms**.

2.3 La scala mel

La percezione umana dell'altezza non è lineare in frequenza: distinguiamo bene 200 da 300 Hz, quasi nulla 8000 da 8100 Hz. La scala mel modella questa compressione:

$$m(f) = 2595 \cdot \log_{10}(1 + f / 700) \quad (2.2)$$

Si costruisce quindi un banco di M filtri triangolari equispaziati in *mel*, e si proietta lo spettro di potenza $|X[m, k]|^2$ su questa base. Il risultato è una matrice $M \times T$ molto più compatta dello spettrogramma lineare, con risoluzione fine alle basse frequenze — dove stanno formanti e armoniche della voce — e grossolana alle alte.

2.4 Il logaritmo

Si applica infine il logaritmo alle energie di banda. Tre ragioni, tutte solide:

- **percettiva**: la sonorità percepita è approssimativamente logaritmica nell'intensità (legge di Weber-Fechner);
- **statistica**: le energie hanno distribuzione fortemente asimmetrica con code lunghe; il logaritmo le rende quasi gaussiane, che è il regime in cui le reti si addestrano meglio;
- **algebrica**: un guadagno moltiplicativo sul segnale (registrare più forte) diventa un *offset additivo* costante sullo spettrogramma logaritmico. Questo trasforma un fastidio moltiplicativo in uno additivo, che una normalizzazione o una BatchNorm rimuove banalmente.

2.5 I nostri parametri, e perché EVIDENZA

Il paper **non dichiara alcun parametro dello spettrogramma**: né il tipo, né `n_fft`, né l'*hop*, né il numero di bin. Avevamo identificato questo come il grado di libertà potenzialmente più impattante dell'intera riproduzione. Lo abbiamo misurato.

Esperimento (sonda lineare). Per ciascun valore di M si calcolano le log-mel di 12 000 clip di training e 4 000 di validation, si mediano su 8 finestre temporali, si standardizzano e si addestra una *regressione logistica multinomiale* — un solo strato lineare. L'accuratezza risultante misura quanta informazione *linearmente decodificabile* contiene la rappresentazione, senza coinvolgere alcun modello profondo.

bin mel	dim. feature	accuratezza sonda	F dopo lo stem
32	256	41,60 %	8
40	320	41,93 %	10
64	512	42,18 %	16
80	640	42,40 %	20
128	1024	42,03 %	32

IL RISULTATO SMENTISCE LA PREOCCUPAZIONE INIZIALE

L'escursione fra il peggiore e il migliore è di **0,8 punti percentuali** su un intervallo di risoluzione che varia di un fattore quattro, e la curva **non è monotona**: 128 bin fanno peggio di 80. Il parametro che temevamo dominante è in realtà quasi **piatto**.

Scegliamo **64 bin**: sta a 0,22 punti dal migliore — differenza dentro il rumore — ed è l'unico valore che dopo lo stem convolutivo produce $F = 16$, una potenza di due. Questo conta perché il modello sparso inizializza le regioni «a metà della dimensione della feature map» e ne campiona l'interno: una griglia 16×51 è geometricamente più pulita di una 20×51 .

Limite dichiarato: la sonda lineare è un *lower bound*. Un modello profondo potrebbe sfruttare la risoluzione in modi che un classificatore lineare non vede. L'evidenza giustifica il non accanirsi, non dimostra l'ottimalità.

Parametri finali: $n_fft = 400$ (25 ms), $hop = 160$ (10 ms), $M = 64$, $f \in [20, 7600]$ Hz. Una clip da un secondo produce $16000/160 + 1 = 101$ frame.

MEL O SPETTROGRAMMA LINEARE? EVIDENZA

Domanda più fondamentale del numero di bin, e per un po' l'avevamo data per scontata. Il paper scrive sempre «*spectrogram*» per il proprio modello; l'unica volta che dice «*Mel-spectrogram*» è descrivendo KWT, un lavoro altrui. Tecnicamente, quindi, non lo dichiara.

Il sostegno alla scelta del mel viene da tre fonti, nessuna delle quali è EAT:

- **KWT** (Berg et al. 2021), il modello più vicino al nostro compito — keyword spotting su Speech Commands con un transformer — usa, per parola dello stesso Kavaki, «a sequence of non-overlapping **Mel-spectrogram** patches»;
- **AST** (Gong et al. 2021), l'altro riferimento citato, opera su spettrogrammi 2D con banco mel;
- EAT stesso, pur non usandolo, riporta che i sistemi di riconoscimento audio si basano «mostly on a **mel-spectrogram** representation» e cita una comparazione sistematica fra rappresentazioni che ne deduce la *superiorità*.

UNA PRECISAZIONE CHE VA FATTA

Sarebbe scorretto giustificare il mel dicendo «lo usa l'altro paper assegnato»: **EAT non usa affatto il mel**. Anzi, la sua tesi centrale è l'opposto — opera sulla forma d'onda grezza e argomenta esplicitamente che «the use of mel-spectrogram comes at the cost of having to carefully adjust the parameters for time-frequency resolution and compression».

La conclusione (usare il mel) resta valida, ma poggia su KWT, su AST e sulla comparazione che EAT cita — non su EAT. All'orale un'affermazione giusta con la motivazione sbagliata è peggio di un dubbio dichiarato.

2.6 Normalizzazione per-campione ASSUNZIONE EVIDENZA

Avevamo aggiunto una normalizzazione che porta ogni spettrogramma a media 0 e deviazione 1. La motivazione era che in Speech Commands il livello di registrazione varia enormemente. **Abbiamo misurato entrambe le cose**: quanto varia, e se la normalizzazione aiuta.

Quanto varia, su 20 000 clip di training:

percentili del livello RMS	p1 -40,6 dB	p25 -29,7 dB
	p50 -24,4 dB	p75 -20,7 dB
	p99 -12,0 dB	

escursione p1-p99 : 28,5 dB -> fattore 27x in ampiezza

La premessa è quindi vera e non di poco: la stessa parola può arrivare al modello con ampiezze che differiscono di oltre un ordine di grandezza.

MA L'EVIDENZA NON SOSTIENE LA NOSTRA AGGIUNTA

Ripetendo la sonda lineare a 64 bin con e senza normalizzazione:

con normalizzazione	: 42,33 %
senza normalizzazione	: 42,83 %

La normalizzazione **peggiora** di mezzo punto. L'interpretazione plausibile è che il livello assoluto porti un po' di segnale utile — o correli con condizioni di registrazione che a loro volta correlano con le classi — e che azzerarlo distrugga questa informazione.

Decisione: default `normalize=False`. La variante resta un flag e diventa una delle ablation dichiarate, non un default silenzioso. A rafforzare la scelta: **né il paper né i due ancestor** — SparseFormer ed EAT, il cui codice abbiamo letto — normalizzano lo spettrogramma. Non abbiamo evidenza a favore di un'aggiunta introdotta per convinzione, e la fedeltà alla riproduzione ha la precedenza.

3 · Il transformer

3.1 L'attenzione come dizionario morbido

Un dizionario classico associa a una chiave esatta un valore. L'attenzione generalizza: data una *query* $\mathbf{q}$, invece di cercare la chiave identica si calcola una **similarità** con tutte le chiavi $\mathbf{k}_j$, la si normalizza a distribuzione di probabilità, e si restituisce la *media pesata* dei valori:

$$\text{Attn}(\mathbf{q}, \mathbf{K}, \mathbf{V}) = \hat{\mathcal{E}}_j \hat{\mathbf{I}}_{\pm j} \mathbf{v}_j, \quad \alpha = \text{softmax}(\mathbf{q}^\top \mathbf{K} / \sqrt{d_k}) \quad (3.1)$$

Nella *self*-attention query, chiavi e valori derivano tutte dalla stessa sequenza tramite tre proiezioni lineari apprese. In forma matriciale, per una sequenza $\mathbf{X} \in \mathbb{R}^{n \times d}$:

$$\mathbf{Q} = \mathbf{X} \mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X} \mathbf{W}_K, \quad \mathbf{V} = \mathbf{X} \mathbf{W}_V, \quad \mathbf{A} = \text{softmax}(\mathbf{Q} \mathbf{K}^\top / \sqrt{d_k}), \quad \text{out} = \mathbf{A} \mathbf{V} \mathbf{W}_O \quad (3.2)$$

3.2 Derivazione dello scaling $1/\sqrt{d_k}$

Perché proprio quel fattore? Si assuma che le componenti di $\mathbf{q}$ e $\mathbf{k}$ siano indipendenti, a media nulla e varianza unitaria. Il prodotto scalare è una somma di d_k termini indipendenti:

$$\text{Var}(\mathbf{q}^\top \mathbf{k}) = \text{Var}(\hat{\mathcal{E}}_{i=1}^{d_k} q_i k_i) = \hat{\mathcal{E}}_i \text{Var}(q_i k_i) = d_k \quad (3.3)$$

La deviazione standard dei logit cresce dunque come $\sqrt{d_k}$. Senza correzione, con $d_k = 64$ i logit hanno scala ± 8 e la softmax entra in saturazione: una componente si avvicina a 1, tutte le altre a 0. Il gradiente della softmax è $\partial \alpha_i / \partial z_j = \alpha_i (\delta_{ij} - \alpha_j)$, che nei regimi saturi tende a zero: il gradiente svanisce e l'attenzione non impara più a riallocarsi. Dividere per $\sqrt{d_k}$ riporta la varianza a 1 indipendentemente dalla dimensione.

3.3 Multi-head

Una singola attenzione produce *una* distribuzione di pesi: il modello è costretto a un unico criterio di rilevanza per posizione. Il multi-head suddivide la dimensione d in h sottospazi di dimensione $d_h = d/h$, esegue l'attenzione in parallelo in ciascuno e concatena:

$$\text{MHSA}(\mathbf{X}) = [\text{head}_1 \parallel \dots \parallel \text{head}_h] \mathbf{W}_O, \quad \text{head}_i = \text{Attn}(\mathbf{X} \mathbf{W}_Q^{(i)}, \mathbf{X} \mathbf{W}_K^{(i)}, \mathbf{X} \mathbf{W}_V^{(i)}) \quad (3.4)$$

Il costo totale è invariato rispetto a una singola testa di dimensione d , perché le proiezioni per-testa sono più piccole: è un guadagno di espressività a parità di calcolo.

IL NUMERO DI TESTE È GRATUITO EVIDENZA

Misurato sul nostro modello con $d = 224$: con 4, 7, 8 o 14 teste, **parametri e FLOPs sono identici** (4 899 151 e 0,5742 G in tutti i casi). La scelta è dunque libera e non ha costo. Adottiamo $h = 7$, che dà $d_h = 32$, la dimensione per testa convenzionale in letteratura (contro i 28 di $h = 8$).

3.4 Complessità — la derivazione centrale del progetto

È la formula che giustifica l'intero paper. Per un blocco, sequenza di lunghezza n in dimensione d , con rapporto FFN r :

Operazione	Costo (MAC)	In n
Proiezioni $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}$	$4nd^2$	lineare
Punteggi $\mathbf{QK}^T$	n^2d	quadratico
Aggregazione $\mathbf{AV}$	n^2d	quadratico
Feed-forward	$2rnd^2$	lineare

$$C(n, d) = (4 + 2r) \cdot n d^2 + 2 \cdot n^2 d \quad (3.5)$$

Il termine quadratico domina quando $n > (2 + r)d$, cioè per $d = 224$ e $r = 4$ quando $n > 1344$.

UNA SOTTIGLIEZZA CHE CAMBIA LA LETTURA DEL PAPER

Le nostre sequenze sono **molto** più corte di quella soglia: 51 frame per il denso, 4 token per lo sparso. In questo regime **domina il termine lineare**, non quello quadratico. Passare da 51 a 4 riduce il termine quadratico di $(51/4)^2 \approx \mathbf{163}$ volte, ma quello lineare — che è quello che conta davvero qui — «solo» di 12,75 volte.

Il paper motiva il lavoro con la complessità $O(N^2)$ della MHSA, ma sul suo stesso regime operativo il guadagno reale viene soprattutto dal termine lineare. Non è un errore, ma è una **imprecisione argomentativa** che vale la pena saper discutere all'orale — ed è la ragione per cui misuriamo i FLOPs invece di dedurli dalla formula.

Parametri, invece, indipendenti da n : $(4 + 2r)d^2$ per blocco, più bias e LayerNorm. Verificato empiricamente: la formula approssima il conteggio reale entro lo 0,4–0,9 %.

3.5 Pre-norm contro post-norm ASSUNZIONE

Vaswani et al. 2017 normalizzano *dopo* il residuo: $x \leftarrow \text{LN}(x + \text{Sublayer}(x))$. Questa forma richiede un warmup del learning rate per non divergere nelle prime iterazioni. La variante **pre-norm** — $x \leftarrow x + \text{Sublayer}(\text{LN}(x))$ — lascia il ramo residuo come strada *pulita* dall'ingresso all'uscita: il gradiente vi scorre senza attraversare alcuna normalizzazione, e il modello si addestra stabile anche senza warmup.

Adottiamo il pre-norm. Il paper non specifica; è pratica standard dal 2020 e riduce il rischio di divergenza in una ricetta che non possiamo tarare a fondo.

3.6 Codifica posizionale e invarianza a permutazioni EVIDENZA

La self-attention è **equivariante alle permutazioni**: permutando le righe di $\mathbf{X}$ si permutano identicamente le righe dell'uscita. Se a valle si applica una media (come facciamo), l'equivarianza diventa **invarianza**: il modello non può in linea di principio distinguere l'ordine dei suoi token.

Il paper dichiara che il modello sparso **non usa codifica posizionale**, e la motivazione è corretta: i token latenti non hanno un ordine intrinseco — sono un insieme, non una sequenza. Sul modello *denso* però il paper tace, e lì i token sono frame temporali, che un ordine ce l'hanno.

Lo abbiamo verificato invece di darlo per scontato. Permutando l'asse temporale di un ingresso e confrontando l'uscita mediata:

senza codifica posizionale : variazione massima $2,4 \cdot 10^{-7}$ (rumore di virgola mobile)
 con codifica posizionale : variazione massima $8,3 \cdot 10^{-2}$

Il primo valore è **invarianza esatta**. Questo rende l'esperimento «con e senza» legittimo e informativo: la domanda diventa *quanto conta l'ordine temporale per riconoscere una parola isolata di un secondo*, che è una domanda scientifica genuina e non una formalità.

Implementiamo tre varianti: assente, sinusoidale (Vaswani) e appresa. La sinusoidale usa

$$PE_{(pos, 2i)} = \sin(pos / 10000^{2i/d}), \quad PE_{(pos, 2i+1)} = \cos(pos / 10000^{2i/d}) \quad (3.6)$$

4 · Convoluzione e early convolution

4.1 Aritmetica

Per un asse di dimensione S , kernel K , stride s , padding p e dilatazione $\hat{l}$:

$$S_{\text{out}} = \lfloor (S + 2p - \delta(K-1) - 1) / s \rfloor + 1 \quad (4.1)$$

Applicata al nostro stem, partendo da 64×101 :

```
conv 7x7 stride 2 pad 3   64 -> 32   101 -> 51
maxpool 3x3 stride (2,1) 32 -> 16   51 -> 51
-----
feature map              [C, 16, 51]
```

Il **campo recettivo** cresce come $R_l = R_{l-1} + (K_l - 1) \cdot \Pi_{i < l} s_i$: dopo conv $7 \times 7 / 2$ e maxpool 3×3 , ogni cella dell'uscita vede 11 frame originali, cioè circa 110 ms di segnale — la scala di un fonema.

4.2 Perché la early convolution è indispensabile DAL PAPER

Il paper è esplicito e la ragione è profonda:

«Gradient computation for region adjustment through bilinear interpolation can be very noisy due to local and limited sampling points. Therefore, directly sampling from the spectrogram is not effective.»

Il campionamento sparso è differenziabile *rispetto alle coordinate* attraverso l'interpolazione bilineare (§7.3). Ma il gradiente rispetto a una coordinata è essenzialmente una **differenza finita locale** della superficie campionata. Su uno spettrogramma grezzo, che ha struttura ad alta frequenza spaziale — armoniche, transienti, rumore — quella differenza è dominata dal rumore locale e non porta informazione utile su *dove* spostare la regione. Dopo qualche convoluzione la superficie è liscia, e il gradiente diventa un segnale di direzione affidabile.

Non è quindi solo una riduzione di risoluzione: è ciò che rende **addestrabile** il meccanismo centrale del paper.

4.3 La sequenza esatta dello stem EVIDENZA

Il paper descrive «conv 7×7 stride 2 $\rightarrow$ ReLU $\rightarrow$ max pooling», senza normalizzazione. Il codice di SparseFormer conferma e completa:

```
conv1 = nn.Conv2d(in, C, kernel_size=7, stride=2, padding=3, bias=True)
relu = nn.ReLU(inplace=True)
maxpool = nn.MaxPool2d(3, 2, 1)

x = conv1(x); x = relu(x); x = maxpool(x)
x = layer_norm_by_dim(x, 1) # LayerNorm sui CANALI, dopo il pooling
```

Tre dettagli che contano:

- la convoluzione ha **bias attivo**, coerentemente con l'assenza di una normalizzazione immediatamente successiva;
- non c'è **alcuna BatchNorm**;
- c'è una **LayerNorm sull'asse dei canali**, applicata *dopo* il max pooling.

UNA NOSTRA DEVIAZIONE, CORRETTA

Una versione precedente del nostro `EarlyConv` inseriva una `BatchNorm2d` fra convoluzione e ReLU, con `bias=False` sulla conv. Era una deviazione su **tre punti**: tipo di normalizzazione sbagliato, posizione sbagliata, bias disattivato.

La differenza fra le due normalizzazioni non è cosmetica. La **BatchNorm** normalizza ciascun canale rispetto al batch e alle posizioni spaziali: le statistiche dipendono dagli altri esempi del batch e vanno congelate in valutazione. La **LayerNorm sui canali** normalizza ogni posizione spaziale rispetto ai propri canali: è indipendente dal batch e si comporta identicamente in training e in inferenza. Per un modello che verrà valutato su costo computazionale e latenza, la seconda è anche più prevedibile.

Coperto ora da un test di regressione che verifica la sequenza esatta dei moduli e l'assenza di `BatchNorm2d`.

5 · Ottimizzazione

5.1 Da SGD ad Adam

La discesa del gradiente stocastica aggiorna $\theta \leftarrow \theta - \eta g_t$ con g_t stima del gradiente su un minibatch. Il **momento** accumula una media mobile della direzione, smorzando le oscillazioni nelle direzioni a curvatura elevata. I metodi **adattivi** aggiungono un secondo ingrediente: riscaldare ogni coordinata in base alla magnitudine storica del suo gradiente, così che parametri con gradienti tipicamente piccoli ricevano passi relativamente maggiori.

Adam combina i due:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2 \quad (5.1)$$

LA CORREZIONE DEL BIAS, DERIVATA

Inizializzando $m_0 = 0$, si espande la ricorsione:

$$m_t = (1-\beta_1) \sum_{i=1}^t \beta_1^{t-i} g_i \quad (5.2)$$

Se i gradienti fossero stazionari con media g , si avrebbe $\mathbb{E}[m_t] = g(1-\beta_1)\sum_{i=1}^t \beta_1^{t-i} = g(1 - \beta_1^t)$. La stima è dunque **distorta verso zero** di un fattore $(1 - \beta_1^t)$, grave nei primi passi: con $\beta_1 = 0,9$ al primo passo il momento vale un decimo del vero. Da cui la correzione:

$$\hat{m}_t = m_t / (1-\beta_1^t), \quad \hat{v}_t = v_t / (1-\beta_2^t), \quad \theta \leftarrow \theta - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) \quad (5.3)$$

5.2 AdamW: perché L2 e weight decay non coincidono

È la domanda d'esame più probabile su questo blocco.

Nella discesa del gradiente *semplice*, aggiungere alla loss il termine $\lambda/2 \cdot \|\theta\|^2$ produce nel gradiente il contributo $\lambda\theta$, e quindi l'aggiornamento $\theta \leftarrow (1 - \eta\lambda)\theta - \eta g$: un decadimento moltiplicativo uniforme. Regularizzazione L2 e weight decay **coincidono**.

In Adam no. Il termine $\lambda\theta$ entra dentro g_t , quindi viene **diviso per $\sqrt{\hat{v}_t}$** come ogni altra componente del gradiente. Il risultato è che il decadimento effettivo di ciascun peso diventa $\eta\lambda\theta/(\sqrt{\hat{v}_t} + \epsilon)$: **i parametri con gradienti storicamente grandi vengono regolarizzati di meno**, quelli con gradienti piccoli di più. È l'opposto di ciò che si vuole, ed è un effetto non voluto e difficile da controllare.

Loshchilov & Hutter (2018) propongono di **disaccoppiarlo**: applicare il decadimento direttamente ai pesi, fuori dal percorso adattivo:

$$\theta \leftarrow \theta - \eta (\hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) + \lambda \theta) \quad (5.4)$$

Entrambi i paper usano AdamW. Noi con $\beta = (0,9 \cdot 0,99)$, $\epsilon = 10^{-8}$, $\eta = 3 \cdot 10^{-4}$, $\lambda = 10^{-5}$ DAL PAPER.

WEIGHT DECAY SELETTIVO: NON È UNA NOSTRA AGGIUNTA EVIDENZA

Avevamo classificato come *nostra deviazione* l'esclusione di bias e parametri di normalizzazione dal weight decay. **Era una classificazione sbagliata.** Il `trainer.py` di EAT costruisce l'ottimizzatore con `weight_decay=0` e delega la regolarizzazione a gruppi di parametri creati da `add_weight_decay`, il cui criterio è:

```
if len(param.shape) == 1: # nessun weight decay
```

Ogni parametro monodimensionale è escluso: bias di convoluzioni e linear, pesi e bias di LayerNorm e BatchNorm. Il criterio è più semplice e più generale di «bias e norm», e li cattura tutti.

Poiché Kavaki & Mandel riprendono la ricetta di EAT alla lettera — stesso lr $3 \cdot 10^{-4}$, stessi β , stesso one-cycle, stesso EMA 0,995, stesso weight decay 10^{-5} , stesse augmentation — la lettura più difendibile è che abbiano ereditato anche questo. Da deviazione a **scelta conforme al riferimento**.

La motivazione teorica resta quella di prima: spingere verso zero un parametro di scala di normalizzazione non ha senso statistico, perché non controlla la complessità del modello ma la sua parametrizzazione interna.

5.3 One-cycle DAL PAPER

Il learning rate sale da $\hat{l}_{\max}/\text{div}$ fino a $\hat{l}_{\max}$ nella prima frazione del training, poi scende fino a un valore molto piccolo. La fase crescente agisce come warmup ed esplora ampiamente il paesaggio; quella decrescente rifinisce. Smith (2018) argomenta che il learning rate elevato in mezzo funge da regolarizzatore, scoraggiando i minimi stretti.

Il paper cita lo schedule ma non ne dà i parametri. Li abbiamo presi dal `trainer.py` di EAT EVIDENZA:

Parametro	Valore	Fonte
<code>max_lr</code>	$3 \cdot 10^{-4}$	dichiarato da entrambi i paper
<code>pct_start</code>	0,1	esplicito nel trainer di EAT — il 10 % del training è salita
<code>div_factor</code>	25	default PyTorch, non sovrascritto da EAT → lr iniziale $1,2 \cdot 10^{-5}$
<code>final_div_factor</code>	10^4	default PyTorch → lr finale $1,2 \cdot 10^{-9}$

Restano non fissate da nessuna fonte le **epoche**: è l'unico iperparametro di training che dobbiamo scegliere noi.

5.4 Media mobile esponenziale dei pesi DAL PAPER

$$\theta^{\text{EMA}} \leftarrow \gamma \theta^{\text{EMA}} + (1-\gamma) \theta, \quad \gamma = 0,995 \quad (5.5)$$

Con $\gamma = 0,995$ la costante di tempo è circa $1/(1-\gamma) = 200$ passi. L'EMA media la traiettoria SGD nelle sue ultime centinaia di iterazioni, riducendo la varianza dovuta al rumore dei minibatch e tendendo a finire in regioni più piatte del paesaggio, che generalizzano meglio. **Il modello che si valuta è quello EMA**: dimenticarlo costa tipicamente qualche decimo di punto.

Verificato nei test: con decay 0,9, spostando i pesi di +1,0 l'EMA si sposta esattamente di $1 - \gamma = 0,1$.

6 · Loss e augmentation

6.1 Il gradiente della cross-entropy con softmax

Sia $\mathbf{z}$ il vettore dei logit, $p_i = e^{z_i} / \sum_j e^{z_j}$ e $L = -\sum_i y_i \log p_i$. Dalla derivata della softmax $\partial p_i / \partial z_j = p_i (\delta_{ij} - p_j)$:

$$\partial L / \partial z_j = \sum_i (-y_i / p_i) \cdot p_i (\delta_{ij} - p_j) = p_j \sum_i y_i - y_j = p_j - y_j \quad (6.1)$$

La forma $\mathbf{p} - \mathbf{y}$ è il motivo per cui softmax e cross-entropy si usano insieme: la composizione dà il gradiente più semplice possibile, e non satura come farebbe una MSE su una sigmoide.

IL TRUCCO LOG-SUM-EXP

Calcolare e^{z_i} direttamente trabocca per $z_i \hat{=} 88$ in float32. Sfruttando l'invarianza della softmax per traslazione dei logit si sottrae il massimo:

$$\log \sum_i e^{z_i} = z_{\max} + \log \sum_i e^{z_i - z_{\max}} \quad (6.2)$$

Tutti gli esponenti sono ora ≤ 0 e nessuno trabocca. È ciò che fanno internamente `log_softmax` e `BCEWithLogitsLoss`: motivo per cui si passa loro i *logit* e non le probabilità.

6.2 Perché BCE con target mixati

Mixing e phasemix producono target del tipo $y = \lambda y_1 + (1-\lambda) y_2$, cioè distribuzioni con due componenti non nulle. La cross-entropy con softmax su un target del genere **non è più il log-likelihood di alcun modello generativo**: la softmax modella una variabile categoriale con esito unico, e l'esempio mixato non ha un esito unico.

Entrambi i paper usano quindi la **binary cross-entropy**, che tratta le 35 uscite come 35 problemi binari indipendenti:

$$L = -(1/C) \sum_c [y_c \log \sigma(z_c) + (1-y_c) \log(1-\sigma(z_c))] \quad (6.3)$$

Qui $y_c \in [0,1]$ è perfettamente sensato come *probabilità marginale* che la classe c sia presente — e nulla obbliga la somma su c a fare 1. È proprio questa libertà che permette il target **multi-hot** di §6.3.4, dove un esempio mixato dichiara la presenza di due parole.

Nel nostro codice la BCE è il **percorso unico**: anche senza augmentation i target sono one-hot in forma $[B, C]$, così il ramo di loss non cambia mai. Verificato nei test: i target contengono solo valori 0 o 1, con una o due classi attive per campione.

6.3 Le due augmentation di Kavaki & Mandel EVIDENZA

Il paper dichiara di usare «mixing and phasemix augmentation [1]», cioè due tecniche di EAT. EAT le descrive con **una riga di testo ciascuna**, senza formule. Le abbiamo ricavate dal codice ufficiale (`datasets/batch_augs.py`), e nessuna delle due è quello che il nome suggerisce.

6.3.1 MIXING NON È IL MIXUP DI ZHANG ET AL.

È il **between-class learning** di Tokozume et al. 2017 — che EAT cita come [14] e che noi avevamo in bibliografia senza collegarlo. I due segnali non vengono interpolati linearmente: vengono bilanciati in base al loro **livello sonoro**, e il risultato viene rinormalizzato in energia.

$$G_i = 10 \log_{10} \mathbb{E}[x_i^2] \quad (\text{livello in dB}) \quad (6.4)$$

$$p = 1 / (1 + 10^{(G_1 - G_2)/20} \cdot (1 - \lambda) / \lambda) \quad (6.5)$$

$$x = (p x_1 + (1-p) x_2) / \sqrt{ p^2 + (1-p)^2 }, \quad y = \lambda y_1 + (1-\lambda) y_2 \quad (6.6)$$

Tre osservazioni sul perché è costruito così:

- **il peso dei dati non è il peso delle etichette:** i campioni si mescolano con p , corretto per il livello, mentre le etichette con λ , che è il rapporto *perceptivo* voluto. Se x_1 è molto più forte di x_2 , servono pesi diseguali per ottenere una miscela percettivamente bilanciata;
- la divisione per $\sqrt{p^2 + (1-p)^2}$ **mantiene costante l'energia attesa**. Senza, il volume della miscela diventerebbe un indizio dell'avvenuta augmentation, e il modello potrebbe imparare a sfruttarlo invece che a generalizzare;
- $\lambda \sim \text{Uniform}[0,1 \cdot 1]$, non una Beta.

6.3.2 PHASEMIX PRESERVA L'AMPIEZZA, NON LA MESCOLO

La descrizione di EAT — «mixes phase components *while preserving amplitude*» — va letta alla lettera: l'ampiezza **non si mescola affatto**. Si prende quella di un campione e solo la fase dell'altro.

$$\mathbf{X} = \text{STFT}(x_1), \quad \varphi = \lambda \angle \mathbf{X}_1 + (1-\lambda) \angle \mathbf{X}_2, \quad x = \text{iSTFT}(|\mathbf{X}_1| \cdot e^{j\varphi}) \quad (6.7)$$

$$\text{peso dell'etichetta} = \lambda \cdot 0,5 + 0,5 \in [0,5 \cdot 1], \quad \lambda \sim \text{Uniform}[0,1] \quad (6.8)$$

IL PESO DELL'ETICHETTA È LA FINEZZA CHE NON SI POTEVA INDOVINARE

Mescolare la sola fase altera il segnale *meno* di quanto faccia mescolare tutto: è giusto che l'etichetta resti più vicina all'originale. Il rimappaggio $\lambda \mapsto \lambda/2 + 1/2$ fa esattamente questo, e garantisce che il peso della classe originale **non scenda mai sotto 0,5**. Nessuna descrizione testuale avrebbe potuto suggerirlo.

Le fasi sono interpolate **sugli angoli**, non sui vettori unitari. È una scelta discutibile — l'interpolazione lineare attraversa la discontinuità a $\pm\pi$ — ma qui conta la fedeltà al riferimento, non il miglioramento del metodo. Una nostra prima versione usava i vettori unitari e mescolava anche le ampiezze: era sbagliata su entrambi i fronti.

6.3.3 CONSISTENZA DELLA STFT: UNA PROPRIETÀ NON OVVIA EVIDENZA

«AMPIEZZA PRESERVATA» VALE SULLA MATRICE, NON SUL SEGNALE

Una matrice complessa arbitraria **non è, in generale, la STFT di alcun segnale reale**. Con finestre sovrapposte — qui al 60 %, finestra 400 e hop 160 — i frame adiacenti condividono campioni e quindi si vincolano a vicenda. Alterare le fasi rompe quel vincolo e rende la matrice *inconsistente*.

La iSTFT esegue overlap-add, che equivale a **proiettare sul sottospazio delle STFT consistenti**. Quella proiezione cambia anche i moduli: misurato, un errore relativo medio di circa il **34 %** ricalcolando la STFT del segnale ricostruito.

Non è un difetto dell'implementazione — è una proprietà del metodo, ed è plausibilmente parte di ciò che lo rende efficace come augmentation: il segnale prodotto non è una semplice ricombinazione dei due originali. È lo stesso fenomeno che rende necessari algoritmi come Griffin-Lim per la ricostruzione da spettrogrammi di sola ampiezza.

Due test fissano entrambi i lati: con $\lambda=1$ la funzione è l'identità entro 10^{-4} (correttezza del percorso STFT→iSTFT), e con $\lambda=0$ la deviazione dei moduli è misurabile ma limitata.

6.3.4 IL TARGET NON È MORBIDO: È MULTI-HOT EVIDENZA

Questa è l'ultima e più sottile delle correzioni. Avevamo scritto che i target sono $\lambda y_1 + (1-\lambda)y_2$, cioè distribuzioni pesate. Il ramo 'bce' di `mix_loss` in EAT fa un'altra cosa:

```
target          = F.one_hot(target, n_classes).float()
target_shuffled = F.one_hot(target_shuffled, n_classes).float() * (lam < 0.9)
one_h_mix       = torch.clamp(target + target_shuffled, max=1)
loss            = self.loss(logits, one_h_mix)
```

$$y = \min(y_1 + y_2 \cdot \mathbb{1}[\lambda < 0,9], 1) \quad (6.9)$$

Tre osservazioni, tutte importanti:

- **λ non è un peso, è una soglia.** Non compare nel valore del target ma solo nella decisione se includere o no la seconda etichetta. Se $\lambda \geq 0,9$ il secondo suono è troppo debole per essere udibile e la sua etichetta viene scartata.
- **Il target è multi-hot:** entrambe le classi valgono 1, non λ e $1-\lambda$. È la formulazione naturale per la BCE, che tratta ogni classe come una domanda binaria indipendente — «questa parola è presente?» — e la risposta per una miscela udibile di due parole è sì a entrambe.
- Il `clamp` serve al caso in cui i due campioni appartengano alla **stessa classe**, che altrimenti darebbe un target pari a 2.

PERCHÉ LA NOSTRA PRIMA FORMULAZIONE ERA COMUNQUE COERENTE — MA CON L'ALTRO RAMO

EAT ha due modalità. Il ramo 'ce' combina due loss come $\lambda L_1 + (1-\lambda)L_2$, che per linearità della cross-entropy nel target è equivalente a un target morbido $\lambda y_1 + (1-\lambda)y_2$. Il ramo 'bce', quello che si usa col mixing, fa invece il multi-hot con soglia.

Avevamo implementato la semantica del ramo sbagliato. Non è un errore di equivalenza matematica ma di scelta del riferimento, ed è il tipo di dettaglio che si vede solo leggendo il codice.

Nota sull'implementazione: `randperm` ha **punti fissi**, quindi circa $1/B$ dei campioni per batch viene mixato con sé stesso e il clamp riporta il target a una sola classe. È il comportamento del riferimento, non un difetto — ma va saputo, perché fa fallire i test scritti assumendo che tutti i target mixati abbiano due classi attive.

6.3.5 COME SONO PROGRAMMATE LE AUGMENTATION EVIDENZA

Il paper dice solo «we also employed data augmentation techniques, including mixing and phasemix». La logica di dispatch viene da `BatchAugs.__call__`:

```
se augs non vuoto e rand() <= mix_ratio e epoch > epoch_mix:
    scegli UNA augmentation a caso e applicala all'INTERO batch
```

- `mix_ratio` è la probabilità di mescolare, e il **default di EAT è 1** — cioè *sempre*. Avevamo assunto 0,5: sbagliato;
- `epoch_mix` è una soglia di epoca sotto la quale il mixing è disattivato, utile per far partire il training su dati puliti;
- una sola augmentation per batch, scelta a caso, applicata a **tutto** il batch — non a un sottoinsieme di campioni.

7 · Il metodo sparso, per intero

7.1 L'idea

Invece di dare al transformer tutti i frame temporali, gli si danno N **token latenti**, ciascuno accoppiato a una **regione** del piano tempo-frequenza da cui estrae informazione. Token e regioni iniziali sono **parametri del modello**, appresi: il modello impara *dove guardare*, non solo *cosa farne*. Il processo si ripete L_{rep} volte, raffinando a ogni giro regioni, punti campionati e token.

Configurazione dichiarata: $N = 4$ token, $P = 36$ campioni per token, $d_{\text{token}} = 64$, $d_{\text{enc}} = 128$, $L_{\text{rep}} = 3$, $L_{\text{enc}} = 8$

[DAL PAPER](#).

INIZIALIZZAZIONE EVIDENZA

Il paper dice: «we initialize the center of the regions to a grid and the width and height of them to half of the feature dimension». Il codice di SparseFormer dà la forma precisa:

```
nn.init.trunc_normal_(token_embedding.weight, std=1.)

root = sqrt(N)
grid = arange(root) / (root - 1)          # [0, 1] uniforme
token_roi[..., 0:2] = grid * (1 - unit_width) # angolo in basso a sinistra
token_roi[..., 2:4] = grid * (1 - unit_width) + unit_width
```

I token si inizializzano da una normale troncata con deviazione 1; le regioni su una griglia regolare in coordinate normalizzate $[0, 1]$, con lato `unit_width` (0,5 secondo il paper). Le regioni sono memorizzate come $(\mathbf{x}_1, \mathbf{y}_1, \mathbf{x}_2, \mathbf{y}_2)$ — angoli opposti — e centro e dimensione si ricavano al volo; le equazioni (7.2) restano identiche.

UNA CONSEGUENZA CHE SPIEGA UN DETTAGLIO DEL PAPER

La griglia è $\text{root} \times \text{root}$ con $\text{root} = \sqrt{N}$. Da qui segue che **N deve essere un quadrato perfetto** — ed è esattamente quello che si osserva nella Tabella 3 del paper, la cui ablation usa $N \in \{4, 9, 16, 25, 36\}$. Non è una scelta estetica degli autori: è un vincolo strutturale ereditato da SparseFormer.

È il tipo di dettaglio che, notato e spiegato all'orale, dimostra di aver capito il metodo e non solo letto il paper.

7.2 Passo 1 — aggiustamento delle regioni

Una regione è $b = (x, y, w, h)$. Il token produce quattro numeri tramite uno strato lineare, e questi aggiornano la regione:

$$(t_x, t_y, t_w, t_h) = \text{Linear}(t) \quad (7.1)$$

$$x' = x + t_x \cdot w, \quad y' = y + t_y \cdot h, \quad w' = w \cdot e^{t_w}, \quad h' = h \cdot e^{t_h} \quad (7.2)$$

PERCHÉ L'ESPONENZIALE — DOMANDA D'ESAME QUASI CERTA

Tre ragioni, tutte necessarie.

Positività. Larghezza e altezza devono restare positive. $w \cdot e^t$ lo garantisce per costruzione, per qualunque valore reale prodotto dalla rete; una forma additiva $w + t$ richiederebbe un clamp, che azzerà il gradiente quando è attivo.

Invarianza di scala. L'aggiornamento è moltiplicativo: lo stesso t_w raddoppia una regione piccola e una grande. La rete apprende *rapporti*, non incrementi assoluti, ed è la parametrizzazione naturale per una quantità positiva.

Simmetria. t_w e $-t_w$ producono fattori reciproci: allargare e restringere costano lo stesso nello spazio dei parametri. Con la forma additiva no.

Anche gli spostamenti sono *relativi* alla dimensione della regione ($t_x \cdot w$): una regione grande si sposta di più a parità di segnale. Tutto questo viene da Faster R-CNN.

7.3 Passo 2 — campionamento sparso

Ogni token genera P offset relativi, poi mappati in coordinate assolute dentro la propria regione:

$$\{(\hat{I}^x x_i, \hat{I}^y y_i)\}_{i=1..P} = \text{Linear}(\text{LayerNorm}(t)) \quad (7.3)$$

$$\tilde{x}_i = x + 0,5 \cdot \Delta x_i \cdot w, \quad \tilde{y}_i = y + 0,5 \cdot \Delta y_i \cdot h \quad (7.4)$$

Il fattore 0,5 mappa $\Delta \in [-1, 1]$ esattamente sull'estensione della regione.

LA NORMALIZZAZIONE «A TRE DEVIAZIONI STANDARD» EVIDENZA

Il paper scrive: «To make sure that most of the sampled points fall inside of the region, we need to do normalization over the relative offsets by three standard deviations», senza formula. Il codice di SparseFormer la fornisce:

```
offset_mean = relative_offset_xy.mean(-2, keepdim=True)
offset_std = relative_offset_xy.std(-2, keepdim=True) + 1e-7
relative_offset_xy = (relative_offset_xy - offset_mean) / (3 * offset_std)
```

Media e deviazione sono calcolate **sull'asse dei P punti**, cioè fra i campioni di uno stesso token, non sul batch. L'operazione è quindi una standardizzazione dell'*insieme* di offset generati, seguita da una divisione per 3.

Il senso è geometrico: dopo la standardizzazione gli offset hanno media 0 e deviazione 1; dividendo per 3 la deviazione diventa $1/3$, e per una distribuzione approssimativamente normale circa il **99,7 %** della massa cade in $[-1, 1]$ — cioè, dopo il riscaldamento di (7.4), dentro la regione. È la regola dei tre sigma, applicata per costruzione invece che sperando che accada.

Nota implementativa: standardizzare fra i punti significa che il modello controlla la *forma* della nuvola di campionamento, mentre la sua posizione e dimensione restano governate dalla regione. I due meccanismi sono disaccoppiati, il che spiega perché servano entrambi.

INTERPOLAZIONE BILINEARE E SUO GRADIENTE

Le coordinate sono continue, la feature map è discreta. Detto $(\tilde{x}, \tilde{y})$ con parte intera (x_0, y_0) e frazionaria (a, b) :

$$F(\tilde{x}, \tilde{y}) = (1-a)(1-b)F_{00} + a(1-b)F_{10} + (1-a)b F_{01} + ab F_{11} \quad (7.5)$$

È il meccanismo che rende **differenziabile la selezione**. Derivando rispetto alla coordinata:

$$\partial F / \partial \tilde{x} = (1-b)(F_{10} - F_{00}) + b(F_{11} - F_{01}) \quad (7.6)$$

cioè una **differenza finita locale** fra celle adiacenti. Da qui discendono due fatti fondamentali:

- il gradiente vede solo le **4 celle adiacenti**: è cieco a tutto ciò che sta oltre un pixel di distanza, quindi la regione può muoversi solo per piccoli passi seguendo la pendenza locale;
- se la superficie è ruvida, quella differenza è **rumore**. È esattamente la ragione per cui serve la early convolution (§4.2). Il legame fra i due paragrafi è il cuore tecnico del paper.

7.4 Passo 3 — decoding adattivo

Ora si hanno P vettori di feature campionati, $x^{(0)} \in \mathbb{R}^{P \times C}$, e vanno compressi nel token. Il paper nota che «simply using a linear layer for this encoding is not effective». La soluzione, ereditata da AdaMixer, è generare i pesi di mixing **dal token stesso**:

$$[\mathbf{M}_c \mid \mathbf{M}_s] = F(t), \quad \mathbf{M}_c \in \mathbb{R}^{C \times C}, \quad \mathbf{M}_s \in \mathbb{R}^{P \times P} \quad (7.7)$$

$$x^{(1)} = \text{GELU}(x^{(0)} \mathbf{M}_c), \quad x^{(2)} = \text{GELU}(\mathbf{M}_s x^{(1)}), \quad t' = t + \text{Linear}(x^{(2)}) \quad (7.8)$$

LA DIFFERENZA CONCETTUALE FRA PESI FISSI E PESI CONDIZIONATI

Uno strato lineare ordinario applica **gli stessi pesi** a tutti gli input: impara una trasformazione media, buona in media per tutti i token. Qui invece $\mathbf{M}_c$ e $\mathbf{M}_s$ sono **funzioni del token**: ogni token specifica come vuole mescolare i propri campioni. Un token che sta seguendo una regione di bassa frequenza può pesare i canali diversamente da uno su un transiente.

È lo stesso principio delle *hypernetwork* e dei meccanismi di attenzione: la rete genera parte dei propri pesi in funzione del contesto. Il costo si vede nel conteggio dei parametri, perché F deve produrre $C^2 + P^2$ valori.

L'aggiornamento del token è **residuo** ($t' = t + \dots$), come nei blocchi transformer: il token accumula informazione a ogni ripetizione invece di essere riscritto, e il gradiente ha un percorso diretto verso i parametri iniziali.

7.5 Il transformer di classificazione

Gli N token finali entrano in 8 blocchi encoder, **senza codifica posizionale**, poi media e classificatore lineare. Con $N = 4$ la matrice di attenzione è 4×4 : praticamente gratuita.

7.6 Contare i FLOPs di questo modello

ATTENZIONE: GRID_SAMPLE NON VIENE CONTATO

L'interpolazione bilineare è un'operazione di *gather* più combinazione lineare: non contiene alcun prodotto matriciale, quindi **né il contatore nativo di torch né fvcore la vedono**. Sul modello sparso questo significa sottostimare proprio il meccanismo che si vuole difendere.

Va aggiunta a mano. Per punto e canale servono 4 letture pesate e 3 somme, più circa 4 operazioni per i pesi condivisi fra canali: `grid_sample_flops(P, C) = P * (4 + 7C)`. È implementata in `src/flops.py`.

8 · La ricostruzione del modello denso

8.1 Il problema

Del dense-transformer il paper fornisce **due soli numeri**: 4,80 M parametri e 0,645 G FLOPs. Nessun iperparametro. Assemblare secondo la Tabella 1 — che descrive lo *sparso* — dà circa 1,8 M: un fattore 2,7 di scarto.

8.2 Il metodo: i numeri come vincoli

Quei due numeri sono **vincoli**. Non tutte le combinazioni delle scelte libere li soddisfano, quindi si può cercare nello spazio delle configurazioni e tenere quelle compatibili — un lavoro di minuti di CPU, senza addestrare nulla. I gradi di libertà considerati:

Grado di libertà	Valori esplorati
Lettura dei canali	<code>c96</code> · <code>c196</code> · <code>c196_proj96</code> (196 kernel poi proiezione a 96)
Stem	<code>paper</code> (conv 7×7 singola) · <code>xiao</code> (stack 3×3)
Stride del pooling	(2,2) → $N=26$ · (2,1) → $N=51$
Formazione della sequenza	<code>flatten</code> · <code>pool_freq</code> · <code>patches2d</code>
Dimensione d	128 ... 288
Profondità	$6 \cdot 8 \cdot 12$

8.3 Risultato 1 — il pooling agisce solo sulla frequenza EVIDENZA

Su 432 configurazioni, **tutte quelle vicine al costo dichiarato hanno $N = 51$** . La famiglia $N = 26$ si ferma a 0,451 G contro 0,645, uno scarto del 30 % che nemmeno lo stem convolutivo più pesante recupera.

Il paper dice solo «max pooling», senza stride. Ne deduciamo che **deve preservare la risoluzione temporale** — un dettaglio non scritto da nessuna parte, ricavato da due numeri.

8.4 Risultato 2 — il paper riporta FLOPs, non MAC EVIDENZA

Ogni configurazione che si avvicina a 0,645 lo fa nella colonna *FLOP* (0,53–0,66); i MAC corrispondenti stanno a 0,26–0,33, cioè metà. Il paper usa dunque la convenzione del contatore nativo di torch.

Il rapporto esatto di 2 fra le due convenzioni è verificato da un test dedicato: su un `Linear(100→200)` senza bias, fvcore riporta 20 000 e torch 40 000. Questa calibrazione rende legittimi tutti i confronti successivi con i numeri pubblicati.

8.5 Un errore di metodo, e la sua correzione

AVEVAMO LASCIATO LIBERA LA PROFONDITÀ

La prima ricerca trattava L come incognita e sceglieva $L = 6$, che fitta i parametri a +0,0 %. Ma il paper dichiara **8 transformer-encoder** per lo sparso e descrive il denso come «*the same architecture but processes a sequence of time frames*»: la profondità **era data**.

Ottimizzare un parametro già vincolato è l'errore classico del fitting. Imponendo $L = 8$ si perdono due punti percentuali sui parametri e si guadagna coerenza col testo — che all'esame vale incomparabilmente di più.

8.6 Un'ipotesi formulata e confutata EVIDENZA

Restava uno scarto di circa l'11 % sul costo. Ipotesi: il paper scrive «convolutional layers» al plurale, quindi gli strati sono più di uno e il costo mancante viene da lì. **Testata:**

n. conv	parametri	errore	GFLOP	errore
1	4 899 151	+2,1 %	0,574	-11,0 %
2	5 245 287	+9,3 %	1,703	+164 %
3	5 591 423	+16,5 %	2,831	+339 %

Ipotesi confutata, e nettamente. Una convoluzione 3×3 con 196 canali alla risoluzione piena (32×51) costa da sola più di 1 GFLOP: aggiungere strati non colma lo scarto, lo travolge. Lo scarto dell'11 % resta **non spiegato**, e va riportato come tale.

8.7 La configurazione adottata

lettura canali	c96	"96 dimensional feature"; il 196 e' un refuso (vedi 12.7)		
stem	paper	una conv 7x7 stride 2, ReLU, maxpool		
pool stride	(2, 1)	solo sull'asse frequenza -> N = 51		
sequenza	pool_freq	media sulla frequenza, N = T		
dimensione	d = 224			
profondita'	L = 8	dichiarata dal paper		
teste	h = 7	d_h = 32		

parametri	4.875.235	dichiarati 4,80 M	->	+1,6 %
costo	0,5275 GFLOP	dichiarati 0,645	->	-18,2 %

Si addestra **anche** la variante `flatten`, che fitta meglio il costo (-5,9 %) ma peggio i parametri: le due bracketizzano l'incertezza. `pool_freq` resta la scelta meno sicura di tutte, perché media via la struttura in frequenza proprio nel baseline che verrà confrontato con un modello che campiona nel piano tempo-frequenza.

PERCHÉ CI FERMIAMO QUI

Nessuna configurazione soddisfa *entrambi* i vincoli, e i due tirano in direzioni opposte. Continuare ad aggiungere gradi di libertà finché i numeri combaciano significherebbe fare **overfitting su una cifra riportata senza convenzione dichiarata**, in un paper che nello stesso paragrafo si contraddice. Un accordo ottenuto così non sarebbe una verifica ma una tautologia.

La posizione difendibile è: parametri entro il 2 %, costo entro l'11 %, un'ipotesi sul residuo formulata e confutata, tutto dichiarato.

9 · Catalogo delle assunzioni

Ogni riga è un punto su cui il paper tace o si contraddice. La colonna «stato» dice se abbiamo evidenza a supporto.

Punto	Scelta	Stato
Parametri spettrogramma	log-mel, 25/10 ms, 64 bin	EVIDENZA sonda lineare: escursione di 0,8 punti su un fattore 4 di risoluzione, curva non monotona. 64 bin sono a 0,22 punti dal migliore e danno $F=16$ dopo lo stem.
Normalizzazione per-campione	disattivata ; flag per ablation	EVIDENZA contraria : la sonda peggiora di 0,5 punti, benché l'escursione di livello nei dati sia di 28,5 dB. Né il paper né i due ancestor la usano.
Normalizzazione nello stem	LayerNorm sui canali dopo il pooling	EVIDENZA sequenza esatta dal codice di SparseFormer. La BatchNorm che avevamo inserito era una deviazione su tre punti: layer, posizione e bias della conv.
Formula di <code>mixing</code>	between-class con correzione di potenza	EVIDENZA dal codice di EAT. Non è il mixup di Zhang et al., come il nome faceva supporre.
Formula di <code>phasemix</code>	STFT, ampiezza preservata, fasi sugli angoli, peso $1/2+1/2$	EVIDENZA dal codice di EAT. La nostra prima versione, dedotta dal testo, era sbagliata su cinque dettagli su cinque.
Parametri STFT di <code>phasemix</code>	400 / 160, come il front-end	ASSUNZIONE EAT non li dichiara.
Normalizzazione degli offset «a tre deviazioni standard»	standardizzazione sull'asse dei punti, poi divisione per 3	EVIDENZA formula esatta dal codice di SparseFormer.
Inizializzazione di token e regioni	trunc-normal std 1; griglia $\sqrt{N} \times \sqrt{N}$	EVIDENZA dal codice di SparseFormer. Spiega perché nel paper N è sempre un quadrato perfetto.
Contraddizione 96 / 196	c96	EVIDENZA risolta dai FLOPs del modello sparso (§12.7): <code>c196_proj96</code> li sbaglia del +73 %, <code>c96</code> tiene tutti e quattro i vincoli entro il 18 %. «96 dimensional feature» è vero, «196 kernels» è il refuso.
Limite sul delta logaritmico delle regioni	<code>MAX_LOG_SCALE</code> = 4	ASSUNZIONE nostra aggiunta, assente dal paper e da SparseFormer. Rete di sicurezza contro l'overflow di <code>exp()</code> ; verificato che non si attivi mai nel regime normale.
Stride del max pooling	$(2, 1) \rightarrow N = 51$	EVIDENZA forzato dal vincolo di costo; la famiglia $N=26$ sbaglia del 30 %.
Formazione della sequenza	<code>pool_freq</code> + <code>flatten</code>	ASSUNZIONE la meno sicura. Si addestrano entrambe.
Numero di teste	$h = 7, d_h = 32$	EVIDENZA parametri e FLOPs identici per 4/7/8/14 teste: scelta libera, si prende la convenzionale.
Pre-norm	pre-norm	ASSUNZIONE standard dal 2020; evita il warmup.
Codifica posizionale nel denso	da abitare	EVIDENZA l'invarianza a permutazioni è verificata: la domanda è ben posta.

Punto	Scelta	Stato
Weight decay su parametri 1D	escluso	EVIDENZA criterio <code>len(shape)==1</code> dal trainer di EAT. Non una nostra aggiunta, come avevamo classificato.
Parametri di one-cycle	<code>pct_start 0,1; div 25; final_div 10</code>	EVIDENZA dal trainer di EAT (gli ultimi due sono i default PyTorch, non sovrascritti).
Programmazione delle augmentation	<code>mix_ratio = 1</code> , una a caso su tutto il batch	EVIDENZA da <code>BatchAugs.__call__</code> . Avevamo assunto probabilità 0,5: il default è <i>sempre</i> .
Costruzione del target mixato	multi-hot con soglia $\hat{y} < 0,9$	EVIDENZA ramo <code>'bce'</code> di <code>mix_loss</code> . Avevamo implementato la semantica del ramo <code>'ce'</code> .
Mel o spettrogramma lineare	mel	EVIDENZA KWT e AST, più la comparazione citata da EAT. <i>Non</i> giustificato da EAT, che usa la forma d'onda grezza.
Label smoothing	assente	ASSUNZIONE EAT usa 0,1 nel ramo CE; con la BCE multi-hot non si applica e Kavaki tace.
Numero di epoche	da fissare	ASSUNZIONE l'unico iperparametro di training che nessuna fonte fissa.

COME È CAMBIATO QUESTO CATALOGO

Prima di leggere il codice degli ancestor questa tabella era quasi tutta **ASSUNZIONE**, e tre delle nostre scelte erano deviazioni non riconosciute. Le due letture successive di `SparseFormer`, `batch_augs.py`, `trainer.py` e `helper_funcs.py` hanno **convertito nove assunzioni in evidenze**, corretto quattro implementazioni sbagliate e **riabilitato una scelta** che avevamo erroneamente classificato come deviazione.

Resta una sola assunzione senza appiglio in nessuna fonte: **il numero di epoche**. Tutto il resto è o dichiarato dal paper, o documentato nel codice di riferimento, o supportato da una nostra misura.

9.1 Che cosa non sappiamo, e non sapremo

Onestà intellettuale richiede di separare ciò che abbiamo risolto da ciò che resta genuinamente aperto.

Questione aperta	Stato
Lo scarto dell'11 % sui FLOPs	Non spiegato. Eliminati due candidati: strati convolutivi aggiuntivi (danno +164 %) e il front-end tempo-frequenza (vale lo 0,8 % del modello). Restano combinazioni di d e L diverse o una formazione della sequenza diversa. Si riporta come tale.
<code>pool_freq</code> contro <code>flatten</code>	Indecidibile dai vincoli: la prima fitta meglio i parametri, la seconda il costo. Si addestrano entrambe e si riportano affiancate.
Pre-norm contro post-norm	Il paper non descrive l'interno degli encoder. Cita Vaswani, che è post-norm; noi usiamo pre-norm. Reso configurabile, da ablare.
Numero di epoche	Nessuna fonte. Si fissa un budget, si dichiara, si verifica la convergenza sulle curve.
Consistenza dei numeri dello sparso	Il conteggio dei parametri torna a -0,8 %, ma i FLOPs dello sparso (0,055 G) non sono ancora stati verificati: è il terzo vincolo indipendente e testerebbe la deduzione <code>pool_stride=(2,1)</code> , che il conteggio dei parametri non tocca. Rimandato alla Fase 4.

10 · Il codice, file per file

10.1 `src/data/speech_commands.py`

Download, verifica MD5, estrazione, costruzione degli split ufficiali e della cache.

La funzione centrale è `build_splits()`, che **solleva un'eccezione** se i conteggi non sono esattamente 84 843 / 9 981 / 11 005. Non è pedanteria: sono i numeri dichiarati dal paper e coincidono con gli elenchi ufficiali `validation_list.txt` e `testing_list.txt`, costruiti sull'hash dello speaker ID e quindi **speaker-disjoint**. Uno split casuale mette lo stesso parlante in train e test e gonfia l'accuratezza di punti interi. Poiché il paper dichiara i tre numeri, abbiamo un test di correttezza gratuito e lo rendiamo bloccante.

`build_cache()` materializza ogni split in un `.npy int16` di forma $[N, 16000]$: 2,72 GB per il training, ~3,4 GB in totale. Le clip più corte di un secondo (l'**11,6 %** del dataset) vengono zero-paddate a destra.

10.2 `src/data/dataset.py`

Dataset e DataLoader sulla cache. Restituisce forme d'onda float32 in $[-1, 1]$; spettrogramma e augmentation avvengono sul batch *in GPU*.

LA MEMMAP NON ATTRAVERSA IL CONFINO DI PROCESSO

Su Windows il DataLoader crea i worker con *spawn*, che **pickla l'oggetto Dataset** per inviarlo a ogni processo. Con la memmap come attributo dell'istanza, pickle tenta di serializzare 2,7 GB dentro una pipe e fallisce con `OSError: [Errno 22] Invalid argument`.

Soluzione: apertura pigra tramite `@property` e `__getstate__` che rimuove l'handle dallo stato serializzato. Ogni worker riapre la propria vista sullo stesso file, e il sistema operativo condivide le pagine fisiche — che è il comportamento voluto. Coperto da un test di regressione che verifica che il blob pickle resti sotto i 100 KB.

Da ricordare: la cache è scritta **in ordine di classe** (34 cambi di etichetta su 84 843 campioni). Ogni iterazione senza shuffle produce quindi batch a classe singola — innocuo in valutazione, disastroso in training.

10.3 `src/data/features.py`

`LogMelSpectrogram`: waveform $[B, 16000] \rightarrow [B, 1, 64, 101]$. Vive sul device del modello. Normalizzazione per-campione opzionale (§2.6).

10.4 `src/data/augment.py`

`mixing` e `phasemix` a livello di batch, su GPU, implementate seguendo `datasets/batch_augs.py` di EAT (§6.3). Entrambe accettano un `lam` opzionale per rendere deterministici i test.

`apply_augmentations()` restituisce **sempre** target morbidi $[B, C]$, anche quando non applica nulla: così il percorso di loss è unico e la BCE non ha rami condizionali.

10.5 `src/models/frontend.py`

`EarlyConv`, parametrico su tutte le ambiguità: `channel_reading`, `stem`, `num_conv_layers`, `norm`, `pool_stride`. Espone `output_shape()`, che calcola la geometria dell'uscita *senza eseguire il forward* — usato dalla ricerca sulle configurazioni.

Contiene `ChannelLayerNorm`, che normalizza un tensore $[B, C, H, W]$ sull'asse dei canali: equivale al `layer_norm_by_dim(x, 1)` di SparseFormer. Il default `norm="layernorm"` riproduce la sequenza del riferimento; `"batchnorm"` è la nostra variante e `"none"` la lettura più letterale del paper.

10.6 `src/models/transformer.py`

MHSA, feed-forward, blocco pre-norm, codifica posizionale, stack. Scritto per esteso con `einops`.

Verifica di correttezza: copiando i nostri pesi dentro `nn.MultiheadAttention` di PyTorch, la differenza massima delle uscite è **esattamente 0,0**. Non plausibilità: equivalenza.

10.7 `src/models/dense.py`

Assembla front-end, formazione della sequenza, codifica posizionale, encoder e testa. Tutti i gradi di libertà passano dal costruttore, perché sono oggetto di ricerca e non scelte fatte.

10.8 `src/flops.py`

Due contatori indipendenti — `torch.utils.flop_counter` e `fvcore` — più gli extra manuali per `grid_sample` e la misura di latenza. Le tre avvertenze (FLOPs $\neq$ MAC, `grid_sample` invisibile, misurare in inferenza con batch 1) sono documentate nel file.

10.9 `src/utils.py`

`seed_everything`, `ModelEMA`, `load_config` con risoluzione della chiave `defaults:` (ereditarietà fra configurazioni), e `param_groups` per il weight decay selettivo (criterio `ndim == 1`, §5.2).

10.10 `src/train.py`

Il training loop. Ogni run è determinato dal solo file di configurazione più il seed; la config viene archiviata accanto ai risultati, così è riproducibile senza dover ricordare quali flag erano stati passati.

Struttura di un'epoca: si carica la forma d'onda, si applicano le augmentation *sul batch in GPU*, si calcola il log-mel, si passa al modello, BCE sui target multi-hot, passo di ottimizzazione, passo dello scheduler, aggiornamento EMA. Poi due valutazioni su validation, una sul modello EMA e una sui pesi correnti.

PERCHÉ SI VALUTANO ENTRAMBI I MODELLI

Il numero che conta è quello EMA — è ciò che i paper riportano. Ma valutare anche i pesi correnti costa poco e dice qualcosa: se il divario fra i due si allarga, l'ottimizzazione sta oscillando e l'EMA sta facendo un lavoro di stabilizzazione consistente; se coincidono, la traiettoria è già liscia. È una diagnostica gratuita sul comportamento dell'ottimizzatore.

Scrivi in `results/<tag>/: metrics.csv` per epoca, `best.pt` e `last.pt`, `summary.json`, log TensorBoard. I checkpoint si salvano in modo **atomico** (scrittura su file temporaneo e rinomina), così un'interruzione — su un portatile, per surriscaldamento o chiusura del coperchio — non lascia mai un checkpoint corrotto. `--resume` riparte da `last.pt` ripristinando anche stato dell'ottimizzatore e dello scheduler.

10.11 `src/models/sparse.py`

L'estrattore sparso: i tre passi di §7 come moduli separati e componibili.

- `RegionAdjust` — `Linear(t)` più la parametrizzazione esponenziale di Faster R-CNN. I pesi partono da **zero**, così alla prima iterazione le regioni restano sulla griglia scelta con cura invece di schizzare via. Contiene `MAX_LOG_SCALE`, la rete di sicurezza contro l'overflow di `exp()` (§12.8).
- `SparseSampler` — offset appresi, standardizzazione sull'asse dei P punti, divisione per 3, e `grid_sample` bilineare. Converte da coordinate $[0,1]$ a $[-1,1]$, che è la convenzione di `grid_sample`.
- `AdaptiveDecoder` — $\mathbf{M}_c$ e $\mathbf{M}_s$ generati dal token, doppio mixing con GELU, aggiornamento residuo. L'ultimo `Linear` opera sul tensore appiattito $P \cdot C \rightarrow d$, dedotto dal budget di parametri.
- `SparseFeatureExtractor` — L_{rep} ripetizioni con pesi **non condivisi**, più i parametri appresi `token_init` e `box_init`.
- `init_boxes_on_grid` — griglia $\sqrt{N} \times \sqrt{N}$, che **rifiuta esplicitamente** gli N non quadrati perfetti con un messaggio che spiega perché.

10.12 `src/models/sparse_model.py`

Assembla front-end, estrattore, ponte 64→128, encoder e testa. Espone la **stessa interfaccia del modello denso** — spettrogramma in, logits fuori, più `seq_len` e `feature_shape` — così training loop, valutazione e contatore di FLOPs non li distinguono.

Due metodi diagnostici che non esistono nel denso:

- `diagnostics()` — `region_init_drift` e `region_adjust_norm`, le due misure che rivelano se il meccanismo centrale è inerte (§12.10). Gratuite, nessun forward.
- `sampling_trace()` — regioni e token a ogni stadio, per riprodurre la Figura 2 del paper.

Il buffer non persistente `_box_init_0` conserva la griglia iniziale per misurare lo spostamento: non finisce nei checkpoint e non sporca lo `state_dict`.

10.13 `src/metrics.py`

Accuratezza per classe, accuratezza **bilanciata** (media delle accuratezze per classe, che con 2,6× di sbilanciamento può divergere da quella globale), classi peggiori e coppie più confuse. Scelta deliberata: l'elenco delle coppie invece di una mappa di calore 35×35, che a quella dimensione è quasi illeggibile e dice molto meno (§12.5).

10.14 `src/tracking.py`

CSV, TensorBoard e Weights & Biases dietro una sola interfaccia, con **degrado controllato**: senza wandb, senza login o senza rete il run prosegue e scrive comunque CSV e TensorBoard. Un esperimento da ore non deve morire perché un servizio esterno non risponde. L'id del run si conserva in `wandb_id.txt` così `--resume` riprende la stessa curva. Vedi §11.5.

10.15 `src/__init__.py`

Non è un file vuoto, ed è l'unico posto in cui la cosa che fa abbia effetto: imposta `CUBLAS_WORKSPACE_CONFIG` **prima di qualunque import di torch**. cuBLAS riusa un'area di memoria di lavoro condivisa fra chiamate, e senza questa variabile `use_deterministic_algorithms` non può garantire nulla su `matmul`, `einsum`, `Linear` e sull'intero backward. PyTorch lo segnala con un `UserWarning` per ogni operazione coinvolta — un avviso che scorre nei log senza che nessuno lo noti.

10.16 `scripts/evaluate.py`

Valutazione finale sul test set, con **guardia contro l'uso ripetuto**: scrive `TEST_EVALUATED.json` al primo utilizzo e si rifiuta di ripartire. Con `--force` si può insistere, ma l'evento finisce nello storico. Vedi §11.9.

10.17 `scripts/run_seeds.py`

Esegue la stessa configurazione su più seed, **salta quelli già completati** — quindi è interrompibile e riprendibile — e aggrega media e deviazione standard in un JSON. È il protocollo dei ≥ 3 seed ridotto a un comando.

Attenzione al prefisso: i tag sono `<prefisso>_seed<N>` e il default è il nome della configurazione. Un run precedente con lo stesso nome ma configurazione diversa verrebbe **saltato e aggregato in silenzio**; si usa `--prefix` per separarli.

10.18 `scripts/check_env.py` e `scripts/prepare_data.py`

Il primo verifica i due punti che su questa macchina possono far perdere giornate: la build CUDA giusta per Blackwell (sm_120) e il backend audio. Non si limita a leggere i metadati — lancia un `matmul` reale sulla GPU, che è l'unico modo di sapere che i kernel ci sono per l'architettura giusta.

Il secondo scarica Speech Commands, **verifica gli split e si ferma con errore** se non danno 84 843 / 9 981 / 11 005, poi costruisce la cache memmap.

10.19 `scripts/build_docs.ps1`

Rigenera i tre PDF dai sorgenti HTML in `docs/` usando Edge in modalità headless — preinstallato su Windows, quindi nessuna dipendenza da installare. Contiene annotate le due trappole che sono costate tempo: serve un `--user-data-dir` dedicato, altrimenti un'istanza di Edge già aperta tiene il profilo bloccato e il PDF non viene prodotto *senza alcun messaggio d'errore*; e un flag inventato fa fallire il comando in silenzio.

10.16 `tests/test_pipeline.py`

19 test, nessuno dei quali richiede il dataset. Coprono: costanti del protocollo, forme del front-end, normalizzazione per-campione, invarianti delle augmentation, picklabilità del Dataset, rapporto $2\times$ fra i contatori, comportamento dell'EMA, riproducibilità dei seed.

11 · Protocollo sperimentale

11.1 Dataset e split

Split	Campioni	Uso
train	84 843	addestramento
validation	9 981	scelta degli iperparametri e del checkpoint
test	11 005	valutazione finale, una sola volta

35 classi, 16 kHz, clip zero-paddate a un secondo. Sbilanciamento fra classi nel training: da 1 256 a 3 250 campioni, cioè un fattore 2,6 — non estremo, coerente con l'uso dell'accuratezza semplice come metrica, che è quella riportata dal paper.

11.2 Prestazioni della pipeline dati

throughput ~24 600 campioni/s -> ~3,5 s per epoca di soli dati
picco memoria GPU 59 MB (solo front-end) su 8192 disponibili

La pipeline dati **non sarà mai il collo di bottiglia**: il costo di un'epoca è interamente il modello.

11.3 Costo del training misurato

configurazione	parametri	s/epoca	100 epoche
d=128 L=8	1,79 M	15,6	26,0 min
d=224 L=8	5,19 M	23,7	39,5 min
d=128 L=24	4,96 M	43,8	73,0 min

Le ultime due righe hanno capacità quasi identica ma tempi in rapporto 1,85. A queste dimensioni la GPU è limitata dal **lancio dei kernel**, non dal calcolo: 24 layer significano tre volte le chiamate CUDA su matrici troppo piccole per saturare gli SM. È la dimostrazione misurata, sul nostro hardware, che **i FLOPs non predicono la latenza** — e la ragione per cui, a parità di parametri, preferiamo la larghezza alla profondità.

IL COSTO REALE DEL TRAINING È TRE VOLTE LA STIMA EVIDENZA

La tabella sopra misurava il *solo* encoder su dati sintetici. Il training completo, misurato: **~100 s per epoca**, non 24. Il fattore ~4 viene da quattro contributi che il benchmark non includeva:

- la sequenza reale è di **51 frame**, non 26 — il benchmark usava la geometria del pooling (2,2), prima che deducessimo (2,1);
- il calcolo del **log-mel** a ogni batch;
- le **augmentation**, che per *phasemix* includono una STFT e una iSTFT complete;
- **due** passate di validation per epoca, sul modello EMA e sui pesi correnti.

Anche il picco di memoria sale da 266 MB a **2,7 GB**, sempre ampiamente entro gli 8 GB disponibili.

Budget rivisto: ~2,8 ore per un run da 100 epoche, quindi circa **17 ore** per i sei run della Fase 3 (due varianti di codifica posizionale × tre seed) invece delle 4 stimate.

11.4 La precisione mista non serve EVIDENZA

Ipotesi ragionevole: attivare l'autocast **bf16** dovrebbe ridurre i tempi in modo sensibile. Implementata e misurata su un'epoca completa:

```
fp32:  101,7 s/epoca  val 31,81%  loss 0,2015
bf16:  101,7 s/epoca  val 31,91%  loss 0,2015
```

Nessun guadagno. Non è un errore di configurazione: il `resolved_config.json` del run registra `amp: bf16`, e il contesto di autocast verificato separatamente produce tensori `bfloat16`.

PERCHÉ — ED È COERENTE CON TUTTO IL RESTO

Il modello è **limitato dal lancio dei kernel**, non dall'aritmetica. Con $d = 224$, sequenza 51 e batch 64 le matrici sono troppo piccole perché i tensor core facciano differenza; il tempo se lo prendono l'overhead di lancio CUDA e le parti che non sono moltiplicazioni di matrici — il mel, la STFT e iSTFT di *phasemix*, l'aggiornamento dell'EMA, la misura della norma del gradiente.

È lo stesso fenomeno misurato altre due volte: $d=128$ con 24 layer che costa $1,85 \times d=224$ con 8 layer a parità di parametri, e i FLOPs che non predicono la latenza. **Tre misure indipendenti che convergono sulla stessa spiegazione.**

Che le due epoche coincidano a 0,1 s di distanza non è una coincidenza sospetta ma la conferma della tesi: il tempo è determinato dal *numero* di lanci, identico nei due casi, non dalla precisione con cui si calcola.

Conseguenza: `amp` resta `none`. Il codice è pronto e testato, ma attivarlo aggiungerebbe una variabile numerica in cambio di nulla. La leva per la velocità, se servisse, sarebbe un batch più grande — ma 64 è dichiarato dal paper.

11.5 Tracciamento degli esperimenti

Tre destinazioni dietro una sola interfaccia (`src/tracking.py`), con degrado controllato: se W&B non è installato, non c'è login o cade la rete, **il run prosegue** e scrive comunque CSV e TensorBoard. Un esperimento da tre ore non deve morire perché un servizio esterno non risponde.

- **CSV** — l'unica fonte che sopravvive a tutto, nella cartella del run;

- **TensorBoard** — locale e istantaneo, utile mentre il run gira;
- **Weights & Biases** — il confronto *fra* run, che è ciò che serve con tre seed per configurazione e più configurazioni da mettere a confronto. Con `mode: offline` funziona senza rete e si sincronizza dopo.

Si registrano per epoca: loss di training, **norma globale del gradiente**, learning rate, accuratezza di validation sul modello EMA e sui pesi correnti, secondi per epoca. La norma del gradiente costa nulla — `clip_grad_norm_` con soglia infinita la misura senza tagliare — e una norma che esplode o collassa segnala un problema molto prima che si veda sulla loss.

Il divario fra accuratezza EMA e non-EMA è a sua volta una diagnostica: se si allarga, l'ottimizzazione sta oscillando e la media mobile sta lavorando; se coincidono, la traiettoria è già liscia.

CONFIGURAZIONE DI W&B, E PERCHÉ L'ENTITY RESTA LIBERA

Il tracciamento è **online**: i run si sincronizzano mentre girano, così le curve sono consultabili da qualunque dispositivo senza attendere la fine. Con rete instabile basta `mode: offline`, e si recupera dopo con `wandb sync results/<tag>/wandb/offline-*` — il sync usa il login attivo *in quel momento*, il che permette anche di scegliere l'account dopo aver lanciato.

`entity` è lasciata a `null` di proposito, cioè «l'entity di default di chi è loggato». Fissarla renderebbe il repository **non eseguibile da altri**: chi lo clona non ha accesso alla nostra, e `wandb.init` fallirebbe. Poiché la consegna prevede che il docente possa eseguire il codice, la portabilità ha la precedenza sull'esplicitezza — e l'entity effettiva resta comunque registrata in ogni `summary.json`.

Verificato end-to-end che il `summary` arrivi al server completo dei campi che contano:

```
best_val_accuracy = 0.2747      balanced_val_accuracy
gflops = 0.574231616          gmacs
provenance = {commit, dirty, gpu, torch, platform}
```

Sono le informazioni che rendono un run confrontabile: accuratezza, costo computazionale e stato del codice, tutti nello stesso posto. Senza i FLOPs accanto all'accuratezza, il grafico centrale del progetto — la curva accuratezza contro costo — andrebbe ricostruito a mano sperando che le configurazioni fossero quelle giuste.

11.6 Analisi per classe

A fine training si ricarica il checkpoint migliore e si produce, **su validation**, un rapporto per classe con accuratezza, supporto, classi peggiori e coppie più confuse (`validation_report.json`). Lo stesso avviene sul test set in `scripts/evaluate.py`.

Si riporta anche l'**accuratezza bilanciata**, cioè la media delle accuratezze per classe. Con uno sbilanciamento di 2,6× può divergere sensibilmente da quella globale, e la coppia di numeri dice più di ciascuno dei due.

PERCHÉ LE COPPIE CONFUSE E NON UNA MAPPA DI CALORE

Una matrice 35×35 disegnata è quasi illeggibile. L'elenco delle coppie più confuse è più utile in diagnosi e all'orale: se il modello scambia *seven* con *six* o *forward* con *four* sta facendo errori fonologicamente sensati, e la cosa si commenta; se scambia coppie senza somiglianza acustica, c'è qualcosa che non va nella pipeline. Già dopo una sola epoca di prova le confusioni osservate erano del primo tipo, il che è un segnale che la catena funziona.

11.7 Riproducibilità del resume: tre bug e una lezione

Prima di lanciare i run lunghi abbiamo verificato una proprietà che sembrava già acquisita: *un run interrotto e ripreso deve dare gli stessi numeri di uno mai interrotto*. Non era vero, e scoprirlo ha richiesto tre correzioni.

LA CAUSA RADICE, IN UNA FRASE

Ripristinare lo stato dei generatori casuali non basta se il *numero di consumi* differisce fra i due percorsi.

Il `DataLoader` di PyTorch estrae dal generatore globale in due punti: il seed del `RandomSampler` (quando non gliene si passa uno) e il `base_seed` dei worker. Con `persistent_workers=True` lo fa **una volta per processo**, non una per epoca. Un run ripreso ricostruisce l'iteratore in un processo nuovo e ripete quelle estrazioni: da lì in poi ogni consumatore a valle legge da uno stato sfasato.

#	Consumatore sfasato	Effetto	Correzione
1	seed del sampler	ordine dei dati diverso dopo la ripresa	<code>EpochShuffleSampler</code> : la permutazione è una funzione pura di (seed, epoca)
2	scelta dell'augmentation, che <code>apply_augmentations</code> fa con <code>torch.rand</code> globale	augmentation diverse	<code>seed_epoch</code> : risemina i generatori a ogni epoca
3	<code>base_seed</code> dei worker, consumato <i>dopo</i> la risemina	la correzione 2 era nel posto sbagliato di tre righe	creare l'iteratore prima e seminare dopo

COME È STATA LOCALIZZATA

Confrontare l'accuratezza dopo due epoche non diceva nulla: passava da 0,32 a 0,01 a 0,08 punti senza un andamento leggibile, perché ogni correzione cambia i flussi casuali e quindi la traiettoria — i valori assoluti non sono confrontabili fra tentativi.

La svolta è stata cambiare metrica: confrontare **i pesi del modello** subito dopo la prima epoca ripresa, invece dell'accuratezza dopo due.

```

epoca 1  loss 0.20552399  identica a 8 decimali
epoca 2  loss 0.16076120  identica
epoca 3  divergente su 108/108 tensori

```

Divergenza *immediata* al confine, non accumulo graduale: questo escludeva il rumore numerico e puntava a un input diverso, non a un calcolo diverso.

VERIFICA FINALE

```

tensori diversi: 0/108
loc_full  epoca 3  loss 0.14688354  val 72.6981%
loc_part  epoca 3  loss 0.14688354  val 72.6981%

```

Il resume è ora esattamente trasparente: si può interrompere un run in qualsiasi momento e riprenderlo ottenendo gli stessi numeri. Su un portatile che va in throttling a 77 °C, con run da quasi tre ore, è la differenza fra poter spezzare il lavoro e doverlo rifare.

IL DETERMINISMO, E QUANTO COSTA

Misurato: due run identici in modalità normale differiscono di **0,02 punti**; con `--deterministic` sono **bit-identici**. Il costo è **+10 %** sul tempo per epoca — insolitamente basso, e per la stessa ragione già emersa tre volte: siamo limitati dal lancio dei kernel, non dal calcolo, quindi rinunciare agli algoritmi cuDNN più rapidi pesa poco.

Con differenze fra configurazioni che nel paper valgono 0,21 punti (96,88 contro 96,67), un pavimento di rumore di 0,02 non è trascurabile e crescerebbe su 100 epoche. Azzerandolo, l'unica varianza che resta è quella *fra seed*, che è esattamente ciò che vogliamo misurare e riportare.

11.8 Il determinismo era un avviso, non una garanzia

Al primo lancio vero, con `deterministic=True` già attivo, il log ha rivelato un problema che nessuna delle verifiche precedenti aveva intercettato:

```
UserWarning: Deterministic behavior was enabled ... but this operation is not
deterministic because it uses CuBLAS and you have CUDA >= 10.2. To enable
deterministic behavior you must set CUBLAS_WORKSPACE_CONFIG=:4096:8
```

L'avviso compariva per `matmul`, `einsum` dell'attenzione, `linear` e l'intero backward — cioè ovunque. Usavamo `use_deterministic_algorithms(True, warn_only=True)`: le operazioni non deterministiche *giravano lo stesso*, segnalandolo con una riga di log che scorre via senza che nessuno la noti.

PERCHÉ SERVIVA LA VARIABILE D'AMBIENTE

cuBLAS riusa un'area di memoria di lavoro condivisa fra chiamate. Quando più stream la usano, l'ordine delle riduzioni può variare fra esecuzioni: due somme degli stessi float in ordine diverso differiscono negli ultimi bit. `CUBLAS_WORKSPACE_CONFIG=:4096:8` alloca workspace dedicati e fissa quell'ordine.

Va impostata **prima che torch inizializzi CUDA**, quindi non basta metterla in `train.py`: sta in `src/__init__.py`, il primo file eseguito da qualunque punto d'ingresso — `src.train`, `scripts/evaluate.py`, i test. Così non può essere dimenticata da riga di comando.

Contestualmente il controllo è stato irrigidito a `warn_only=False`: un'operazione priva di implementazione deterministica ora è un **errore**, non un avviso. Il run parte comunque, il che dimostra che nessuna operazione del nostro modello è in quella condizione.

L'ESITO È IL PIÙ INFORMATIVO POSSIBILE

```
tensori diversi: 0/108
epoca 1  loss 0.20624505      identiche a quelle misurate PRIMA
epoca 2  loss 0.16153782      della correzione
epoca 3  loss 0.14688354
```

I numeri non sono cambiati. Il determinismo che avevamo osservato empiricamente era dunque reale — ma reggeva per fortuna, non per costruzione. Su 132 500 passi anziché su tre epoche di prova, quella fortuna non era una base accettabile.

Effetto collaterale misurato: il picco di memoria GPU scende da 2715 a **1087 MB**, perché disattivare l'autotuning di cuDNN elimina i buffer di prova degli algoritmi candidati.

LA LEZIONE, CHE È LA STESSA DI STAMATTINA

Tre bug di riproducibilità erano emersi confrontando percorsi che dovevano coincidere. Questo è emerso **leggendo i log del primo lancio vero**. Nessuna quantità di test avrebbe potuto trovarlo, perché i test passavano: la falla era nella *garanzia*, non nel comportamento osservato.

È il motivo per cui vale la pena leggere gli avvisi invece di lasciarli scorrere.

11.9 Disciplina sperimentale

- **≥ 3 seed** per ogni configurazione riportata, con media e deviazione standard. Motivazione empirica: nella Tabella 3 del paper l'ablation sui token non è monotona (16 → 97,53 %, 25 → 97,20 %, 36 → 97,94 %) su singolo seed, il che suggerisce un rumore di 0,3–0,4 punti, paragonabile a diverse delle differenze discusse.
- **Il test set si tocca una volta sola**, alla fine, sul modello selezionato in validation. La guardia non è un promemoria ma un meccanismo: `scripts/evaluate.py` scrive `TEST_EVALUATED.json` nella cartella del run al primo utilizzo e si rifiuta di ripartire. Con `--force` si può insistere, ma l'evento viene registrato nello storico del file, così resta traccia di quante volte è successo. Motivazione: valutare ripetutamente il test cambiando qualcosa in mezzo equivale a selezionare su di esso, e l'errore *non produce alcun sintomo visibile* — il numero sembra buono, semplicemente non è più una stima onesta della generalizzazione.
- Si valuta il **modello EMA**.
- Con mixing e phasemix sempre attive, **l'accuratezza di training non è interpretabile**: il segnale utile è solo la validation.
- Ogni run è riproducibile dal solo file di configurazione, che viene archiviato accanto ai risultati.

11.10 Target di riproduzione

Modello	Params	FLOPs	Accuratezza	Nota
Sparse-transformer (N=4, P=36)	2,87 M	0,055 G	96,88 %	obiettivo Fase 4
Dense-transformer	4,80 M	0,645 G	96,67 %	obiettivo Fase 3
EAT-S	5,30 M	0,373 G	98,15 %	<i>citato</i> , non rimisurato
HTS-AT	28,80 M	4,066 G	98,00 %	<i>citato</i>
SimPF (MobileNetV2)	3,40 M	0,244 G	95,60 %	<i>citato</i>

12 · Risultati

12.1 Primo run completo — dense-transformer, seed 0

Misura	Nostro	Paper	Scarto
Accuratezza (validation, EMA)	94,67 %	96,67 % [†]	−2,00
Accuratezza bilanciata	94,31 %	—	—
Parametri	4 899 147	4,80 M	+2,1 %
Costo	0,5742 GFLOP	0,645 G	−11,0 %
Picco memoria GPU	1 087 MB	—	—
Tempo	1,80 h (65 s/epoca)	—	—

[†] Il numero del paper è su *test*, il nostro su *validation*: non sono direttamente confrontabili. Il test set non è ancora stato toccato.

QUESTO RUN USA UNA LETTURA DEI CANALI POI SUPERATA

Il run è stato eseguito con `c196_proj96`. La §12.7 mostra che la lettura corretta è `c96`, e per un confronto denso-contro-sparso controllato il front-end deve essere lo stesso nei due modelli: **il denso va rifatto con `c96`**. Il risultato resta valido come primo run completo e come verifica della pipeline, ma non è il riferimento definitivo.

Provenienza registrata nel run: commit `e6106907`, torch 2.9.1+cu128, RTX 5050 Laptop, CUDA 12.8, Windows 11. Determinismo attivo, precisione fp32.

12.2 La curva satura molto prima di 100 epoche EVIDENZA

epoca	lr	val EMA	val raw	EMA – raw
5	$1,56 \cdot 10^{-4}$	81,51 %	77,75 %	+3,76
10	$3,00 \cdot 10^{-4}$	88,89 %	84,60 %	+4,29
30	$2,65 \cdot 10^{-4}$	93,45 %	91,84 %	+1,61
50	$1,76 \cdot 10^{-4}$	94,06 %	93,61 %	+0,45
70	$7,50 \cdot 10^{-5}$	94,49 %	93,97 %	+0,52
90	$9,05 \cdot 10^{-6}$	94,61 %	94,60 %	+0,01
100	$1,20 \cdot 10^{-9}$	94,67 %	94,67 %	0,00

LA CONSEGUENZA PIÙ IMPORTANTE DI TUTTO IL RUN

Dall'epoca 30 alla 100 — cioè nel **70 % del budget di training** — il modello guadagna **1,22 punti**. Dalla 50 alla 100, mezzo run intero, ne guadagna **0,61**. Le ultime dieci epoche hanno un'escursione di **0,06 punti**: la curva è piatta.

Questo **elimina «poche epoche» come spiegazione dei 2 punti mancanti**. Non è un problema di durata dell'addestramento: il modello ha smesso di migliorare molto prima della fine. Il residuo è di natura architetturale o configurativa, e va cercato altrove.

Ha anche una conseguenza pratica notevole per le ablation della Fase 5: **50 epoche darebbero conclusioni identiche a 100** a metà del costo. Con 27 run previsti fra ablation su N e su P, è la differenza fra ~50 e ~25 ore.

12.3 L'EMA si guadagna lo spazio solo ad alto learning rate EVIDENZA

Il divario fra modello EMA e pesi correnti si comporta esattamente come la teoria prevede, ed è una delle illustrazioni empiriche più pulite prodotte dal progetto:

epoca 10:	+4,29 punti	lr al massimo, traiettoria molto rumorosa
epoca 30:	+1,61 punti	lr in discesa
epoca 50:	+0,45 punti	
epoca 90:	+0,01 punti	
epoca 100:	0,00 punti	lr = 1,2e-09, i pesi non si muovono piu'

La media mobile serve a smorzare l'oscillazione dovuta al rumore dei minibatch. Quando il learning rate si annulla, i pesi smettono di muoversi e la media di una traiettoria ferma è il punto stesso: il divario si chiude per costruzione.

Ne segue una nota operativa: **a fine schedule valutare l'EMA o i pesi correnti è indifferente**. L'EMA conta se si interrompe prima, o con schedule che tengono il learning rate alto fino in fondo. Con one-cycle portato a termine, è un'assicurazione che non serve — ma che costa nulla.

12.4 Due correzioni a osservazioni precedenti

LA NORMA DEL GRADIENTE NON CRESCE

Guardando lo *sparkline* di W&B avevo concluso che `train_grad_norm` crescesse lungo il training. **Sbagliato**. I valori dal CSV: 0,26 all'epoca 1, picco di 0,92 alla 5 mentre il learning rate sale, poi discesa e **piatta a ~0,24 dall'epoca 20 fino alla fine**.

Lo *sparkline* è una serie binnata e riscalata: con una metrica quasi costante amplifica il rumore fino a farlo sembrare una tendenza. **Per leggere l'andamento di una metrica si usa il CSV, non la miniatura**. È il motivo per cui il CSV resta la fonte di verità del progetto.

IL PICCO DI MEMORIA È MENO DELLA METÀ DEL PREVISTO

1 087 MB contro i 2 715 misurati negli smoke test. La differenza è il **determinismo**: `cudnn.benchmark = False` impedisce a cuDNN di provare algoritmi alternativi, alcuni dei quali usano molta più memoria di lavoro in cambio di velocità. Si paga in tempo e si guadagna in memoria — su 8 GB nessuna delle due cose è un vincolo.

12.5 Che errori fa il modello

classi peggiori			confusioni piu' frequenti		
forward	86,99%	(146)	off	-> up	10
learn	88,28%	(128)	go	-> down	10
go	90,32%	(372)	tree	-> three	9
backward	90,85%	(153)	go	-> two	9
no	91,38%	(406)	three	-> tree	8
down	91,51%	(377)	forward	-> four	8

Le confusioni sono **tutte fonologicamente motivate**. *tree* ↔ *three* è la coppia minima classica /t/ contro /θ/, ed è simmetrica in entrambe le direzioni — il modello non ha un bias, semplicemente non distingue una fricativa dentale sorda da un'occlusiva alveolare in una parola di 500 ms. *forward* → *four* condivide l'attacco; *go* → *down* e *no* → *go* hanno vocali posteriori simili.

È il controllo qualitativo che conta più dell'accuratezza aggregata: se il modello sbagliasse coppie senza somiglianza acustica, ci sarebbe un difetto nella pipeline dei dati. Qui il profilo di errore è quello di un sistema che ha imparato la fonetica e si ferma dove si fermerebbe un ascoltatore umano su clip brevi e rumorose.

Le classi peggiori sono anche fra le **meno rappresentate** — *forward* 146 campioni, *learn* 128 contro i ~400 delle classi frequenti — ma non solo: *go*, *no* e *down* hanno supporto abbondante e sbagliano lo stesso. Lo sbilanciamento spiega una parte del fenomeno, la somiglianza acustica il resto. Coerente con lo scarto contenuto fra accuratezza globale (94,67 %) e bilanciata (94,31 %).

12.6 Dove cercare i due punti mancanti

Con «poche epoche» escluso da §12.2, restano quattro ipotesi, in ordine di plausibilità:

Ipotesi	Come la si verifica
pool_freq è la scelta sbagliata	È la nostra decisione meno sicura: media via la struttura in frequenza. La variante flatten è già progettata e fitta meglio il costo dichiarato (−5,9 % contro −11,0 %). Un run, 1,8 ore.
Assenza di regolarizzazione	dropout=0 e nessun label smoothing: entrambi non dichiarati dal paper. Un run con dropout 0,1 direbbe se il modello è in sovradattamento — ma la loss di training continua a scendere fino alla fine, quindi è poco probabile.
Differenza validation/test	Parte dello scarto potrebbe non esistere: confrontiamo il nostro validation col loro test. Verificabile solo alla fine, e una volta sola .
Ricostruzione imperfetta del denso	Lo scarto dell'11 % sui FLOPs, mai spiegato, indica che il loro modello non è esattamente il nostro. Non risolvibile senza il loro codice.

12.7 I FLOPs dello sparso sciolgono la contraddizione del paper EVIDENZA

La sezione 3.2 del paper si contraddice: «extract **96 dimensional** feature» ma «the convolutional layers have **196 kernels**». Finora avevamo adottato **c196_proj96** — la lettura in cui entrambe le cifre sono vere — sulla base dei soli vincoli del modello denso. Implementato il modello sparso, i suoi due vincoli separano nettamente le letture:

lettura	DENSO parametri	DENSO costo	SPARSO parametri	SPARSO costo	errore massimo
c96	+1,6 %	−18,2 %	−1,6 %	−11,8 %	18,2 %

lettura	DENSO parametri	DENSO costo	SPARSO parametri	SPARSO costo	errore massimo
c196_proj96	+2,1 %	-11,0 %	-0,8 %	+73,1 %	73,1 %

PERCHÉ È LO SPARSO A DISCRIMINARE, E IL DENSO NO

La early convolution vale l'**11 % del costo del modello denso** e il **65 % di quello sparso**. La ragione è strutturale: lo sparso scarta quasi tutto il calcolo a valle — 4 token invece di 51 frame — e il front-end diventa il termine dominante.

Ne segue che **i due modelli vincolano cose diverse**: i numeri del denso sono dominati dal transformer e fissano d e L , quelli dello sparso sono dominati dal front-end e fissano la lettura dei canali. Usarli insieme è ciò che scioglie l'ambiguità; nessuno dei due da solo ci sarebbe riuscito.

Conclusione: «96 dimensional feature» è l'affermazione vera, «196 kernels» è il refuso — verosimilmente per «96 kernels».

Onestà sul residuo. Con `c96` nessun vincolo è centrato: il costo del denso resta a -18,2 %. Ma un sottostimare *sistematico* su tutti e quattro i vincoli è la firma plausibile di una ricostruzione un filo più leggera dell'originale; un +73 % su un solo modello no. È la differenza fra un errore di scala e un errore di struttura.

12.8 Verifica dell'implementazione sparsa

Misura	Nostro	Paper	Scarto
Parametri	2 822 711	2,87 M	-1,6 %
Costo	0,0485 GFLOP	0,055 G	-11,8 %
<code>grid_sample</code> (contato a mano)	0,29 MFLOP	—	0,3 % del totale

Il conteggio dei parametri era stato **predetto a -0,8 %** sommando i componenti a mano, prima di scrivere una riga di codice (§8). L'implementazione lo conferma. È l'evidenza più forte che la lettura del metodo sia corretta.

Nota sul `grid_sample`: vale lo 0,3 % del totale, quindi il fatto che nessun contatore lo veda è irrilevante *numericamente*. Resta importante contarlo perché è il meccanismo che si sta difendendo, e ometterlo sarebbe una scorrettezza metodologica anche se innocua.

UNA NOSTRA AGGIUNTA: IL LIMITE SUL DELTA LOGARITMICO ASSUNZIONE

Un test con delta patologici ha mostrato che `exp()` in float32 esplode oltre ~88, producendo regioni di dimensione infinita o nulla e quindi NaN nel gradiente. Né il paper né SparseFormer prevedono un limite.

Abbiamo aggiunto `MAX_LOG_SCALE = 4`, cioè al più 55× di crescita per ripetizione e ~163 000× sulle tre — enormemente più di qualunque variazione sensata per regioni che vivono in [0, 1]. Un test dedicato verifica che **nel regime normale il limite non si attivi mai**, quindi non è una deviazione dal metodo ma una protezione contro la perdita di un run di ore.

UN DETTAGLIO DEL CONTATORE DI FLOPS

`FlopCounterMode` falliva sul modello sparso con un `AssertionError` opaco: sotto `no_grad` il suo `ModuleTracker` non riesce a registrare gli hook su un `Parameter` espanso, e le regioni iniziali sono esattamente questo. Si conta con il grafo attivo — il risultato non cambia, perché si contano le operazioni del forward.

12.9 Il metodo sparso non è deterministico su GPU EVIDENZA

Al primo tentativo di addestrare il modello sparso, PyTorch ha interrotto il run:

```
RuntimeError: grid_sampler_2d_backward_cuda does not have a deterministic implementation, but you set 'torch.use_deterministic_algorithms(True)'
```

Non è un difetto della nostra implementazione: `grid_sample` — il campionamento bilineare che è il meccanismo centrale del paper — non ha un backward deterministico su CUDA. Il gradiente rispetto alla feature map accumula i contributi dei P punti campionati con `atomicAdd`, e l'ordine delle somme in virgola mobile cambia da un'esecuzione all'altra.

LA SCELTA DI FAR FALLIRE INVECE DI AVVISARE HA PAGATO

Avevamo impostato `use_deterministic_algorithms(True, warn_only=False)` proprio perché «meglio non partire che scoprire dopo due ore che la garanzia non valeva». Con `warn_only=True` il run sarebbe partito, avrebbe stampato un avviso fra migliaia di righe e prodotto risultati non riproducibili che credevamo bit-esatti.

Risposta adottata: determinismo rilassato a `warn_only` solo per il modello sparso, deciso automaticamente da `model.kind`. Ogni operazione che può restare deterministica lo resta; solo `grid_sample` ricade nel comportamento non deterministico. Il livello effettivo (`strict` / `warn_only` / `off`) viene registrato nel `summary.json`, così un run non bit-riproducibile non si spaccia per tale.

Osservazione da portare all'orale. Il metodo proposto è **intrinsecamente meno riproducibile del proprio baseline denso**, perché il suo meccanismo centrale non è deterministico su GPU. Questo rende l'ablation a singolo seed della Tabella 3 ancora meno solida di quanto già sospettassimo osservandone la non monotonia: gli autori non potevano avere run bit-riproducibili nemmeno volendo.

12.10 Il meccanismo funziona: le regioni si muovono EVIDENZA

Il rischio silenzioso di questo metodo è che le regioni *non* si muovano. Il gradiente rispetto alle coordinate è una differenza finita fra quattro celle adiacenti (eq. 7.6); se non porta segnale utile, le regioni restano dove la griglia le ha messe, il modello degrada a un campionamento fisso, **si addestra comunque e produce un numero plausibile** — senza che nulla nei log lo segnali.

Due misure, entrambe gratuite, registrate a ogni epoca:

Misura	Che cosa dice
<code>region_init_drift</code>	Quanto le regioni <i>apprese</i> si sono spostate dalla griglia iniziale. Zero = il modello non ha imparato dove guardare.
<code>region_adjust_norm</code>	Norma dei pesi che generano l'aggiustamento per-campione. Partono da esattamente zero (inizializzazione all'identità), quindi qualunque valore positivo dimostra che quel ramo riceve gradiente.

Smoke test su due epoche: `region_adjust_norm` passa da 0 a **4,32** dopo una sola epoca. Il meccanismo centrale del paper è vivo. L'accuratezza sale da 35,65 % a 50,59 %.

(Lo *spostamento* resta identico fra le due epoche perché con `--epochs 2 one-cycle` comprime l'intero schedule: alla seconda epoca il learning rate è $1,3 \cdot 10^{-9}$ e nessun parametro si muove più.)

12.11 Il guadagno in FLOPs non si traduce in velocità

EVIDENZA

Modello	FLOPs	s/epoca	picco GPU
Denso	0,5275 G	65 s	1 087 MB
Sparso	0,0485 G (-91 %)	78 s (+20 %)	596 MB

IL MODELLO CHE COSTA UN DECIMO IN FLOPS È PIÙ LENTO DEL 20 %

È la conferma più forte di una tesi che il progetto aveva già misurato per tre vie indipendenti: **questo carico è limitato dal lancio dei kernel, non dall'aritmetica**. L'estrattore sparso sostituisce pochi grandi prodotti matriciali con moltissime operazioni minuscole — tre ripetizioni di einsum per-token, `grid_sample`, `LayerNorm` — e ogni lancio CUDA costa più del calcolo che esegue.

Le altre tre misure che convergono: profondità contro larghezza a parità di parametri (1,85×), bf16 senza alcun guadagno, e i FLOPs che non predicono la latenza sul denso.

Non invalida il paper. La riduzione di FLOPs è reale e verificata, e su un acceleratore per edge device — il caso d'uso che gli autori dichiarano — il profilo di costo può essere completamente diverso. Ma **qualifica il claim in modo sostanziale**, ed è una misura fatta sui due modelli del paper stesso. La riduzione di memoria (−45 %) invece si traduce direttamente.

13 · Riferimenti

PAPER DEL PROGETTO

- Kavaki, H. S., & Mandel, M. I. (2025). *Audio Sparse-Transformer for Speech Classification*. ICASSP 2025, 1–5.
- Gazneli, A., Zimerman, G., Ridnik, T., Sharir, G., & Noy, A. (2022). *End-to-End Audio Strikes Back*. arXiv:2204.11479. Codice: `github.com/Alibaba-MIIL/AudioClassification` — file `datasets/batch_augs.py` per le augmentation di label-mixing.

ANCESTORS DEL METODO

- Gao, Z., Tong, Z., Wang, L., & Shou, M. Z. (2023). *SparseFormer: Sparse Visual Recognition via Limited Latent Tokens*. arXiv:2304.03768. Codice: `github.com/showlab/sparseformer` — file `imagenet/models/sparseformer.py` per la v1.
- Gao, Z., Wang, L., Han, B., & Guo, S. (2022). *AdaMixer*. CVPR 2022.
- Ren, S., He, K., Girshick, R., & Sun, J. (2015). *Faster R-CNN*. NeurIPS 28.
- Xiao, T., et al. (2021). *Early Convolutions Help Transformers See Better*. NeurIPS 34.

FONDAMENTI

- Vaswani, A., et al. (2017). *Attention Is All You Need*. NeurIPS 30.
- Dosovitskiy, A., et al. (2020). *An Image Is Worth 16×16 Words*. ICLR 2021.
- Kingma, D., & Ba, J. (2014). *Adam*. arXiv:1412.6980.
- Loshchilov, I., & Hutter, F. (2018). *Decoupled Weight Decay Regularization*. ICLR 2019.
- Smith, L. N. (2018). *A Disciplined Approach to Neural Network Hyper-Parameters*. arXiv:1803.09820.
- Tarvainen, A., & Valpola (2017). *Mean Teachers Are Better Role Models*. NeurIPS 30.
- Hendrycks, D., & Gimpel, K. (2016). *Gaussian Error Linear Units*. arXiv:1606.08415.
- Dumoulin, V., & Visin, F. (2016). *A Guide to Convolution Arithmetic for Deep Learning*. arXiv:1603.07285.
- Rogozhnikov, A. (2022). *Einops*. ICLR 2022.
- Warden, P. (2018). *Speech Commands*. arXiv:1804.03209.
- Tokozume, Y., Ushiku, Y., & Harada, T. (2017). *Learning from Between-Class Examples*. arXiv:1711.10282.

MODELLI DI CONFRONTO

- Berg, A., O'Connor, M., & Tairum Cruz, M. (2021). *Keyword Transformer*. arXiv:2104.00769.
- Gong, Y., Chung, Y.-A., & Glass, J. (2021). *AST: Audio Spectrogram Transformer*. arXiv:2104.01778.
- Chen, K., et al. (2022). *HTS-AT*. ICASSP 2022.

Manuale tecnico del progetto d'esame Bo31278 — Deep Learning, Fall 2025, Università di Firenze. Documento vivo: sorgente in `docs/manuale_tecnico.html`, rigenerabile con `scripts\build_docs.ps1`. Le formule di §7 sono trascritte dal paper originale; le derivazioni sono nostre. I marcatori distinguono ciò che il paper dichiara, ciò che assumiamo e ciò per cui abbiamo evidenza sperimentale.